

École Polytechnique Fédérale de Lausanne

FPGA Acceleration for Urban Digital Twin Models:
Optimizing Mixed-Integer Linear Programming (MILP)
for Energy Efficiency

by Gaëtan Jenni

Semester Project Report

Prof. David Atienza
Project Advisor

Dr. Denisa-Andreea Constantinescu, Rubén Rodríguez Álvarez
Project Supervisors

STI IEL ESL
ELG 130 (Bâtiment ELG)
Station 11
CH-1015 Lausanne

June 5, 2026

Acknowledgements

I wish to thank my supervisors, Dr. Denisa-Andreea Constantinescu and Rubén Rodríguez Álvarez, for their close mentorship during this project. Their availability for weekly discussions and their detailed feedback were very helpful. I also want to thank them for offering me the opportunity to present intermediate results of this project at the *Sustainability Conference for Responsible Research Computing* (SC4RC 2026) at CERN in May 2026, which was a very great experience.

My interest in high-level synthesis started during the EPFL course *Lab on Hardware-Software Digital Systems Codesign* (EE-390(a)) of David Atienza and Miguel Peón-Quirós. I am grateful to them for the very interesting lectures and for the hands-on labs and project, which I loved doing and which introduced me to the world of HLS.

This project would not have been possible without the support of the Embedded Systems Laboratory (ESL) at EPFL, led by Prof. David Atienza, whom I thank for the opportunity to carry out this project. I am also grateful to the REHO team and the Research Center for Energy Networks (FEN, ETH Zürich) for providing the context and motivation for this project.

Lausanne, June 5, 2026

Gaëtan Jenni

Abstract

Cities increasingly rely on computer models to plan their energy systems. Tools like the Renewable Energy Hub Optimizer (REHO) write these planning tasks as Mixed-Integer Linear Programs (MILPs): powerful, but slow and energy-hungry to solve at large scales. Most of the cost is hidden deep inside the solver, where the same linear-algebra step — factorizing and solving a matrix — runs again and again.

This project speeds up that inner step with an FPGA, a chip that can be tailored to a very specific task. The high-level control of the solver stays on the CPU, while the heavy matrix factorization is offloaded to two custom hardware blocks built with High-Level Synthesis (HLS): one for dense problems, one for sparse ones. Each block is tuned by changing how much work runs in parallel and how many bits represent each number, and these options are swept on a Xilinx ZCU104 board to analyze the trade-offs between runtime, energy, and chip area.

The dense block solves up to $34\times$ faster than the board's own CPU while using about $6\times$ less energy, and stays accurate. The sparse block wins on denser problems (up to $3.7\times$ faster) but not on very sparse ones, where the CPU stays ahead. These results show that accelerating the small but costly kernel at the bottom of an MILP solver could bring real speedups and energy savings on embedded hardware.

Contents

Acknowledgements	2
Abstract	3
List of Acronyms	7
1 Introduction	8
2 Background and Related Work	11
2.1 Mixed-Integer Linear Programming	11
2.2 Interior-Point Method for Linear Programming	12
2.3 Direct Methods for Linear Systems	13
2.3.1 Gaussian Elimination with Partial Pivoting	13
2.3.2 LDL^T Cholesky Factorization	13
2.4 Sparse Linear Algebra	14
2.4.1 Sparse Storage Formats: CSC and CSR	14
2.4.2 Fill-In and Symbolic Factorization	14
2.4.3 Fill-Reducing Ordering (RCM)	15
2.4.4 Simplicial Up-Looking Sparse LDL^T	15
2.5 Fixed-Point Arithmetic	15
2.6 FPGA Platform and High-Level Synthesis	16
2.6.1 FPGA Fundamentals	16
2.6.2 Xilinx ZCU104	16
2.6.3 High-Level Synthesis	17
2.7 Related Work	17
2.7.1 General-Purpose CPU MILP/LP Solvers	17
2.7.2 Decomposition Methods for Energy-System MILP	18
2.7.3 FPGA Acceleration of LP and MILP Solvers	19
3 Design	20
3.1 Computational Bottleneck and Roofline Analysis	21
3.2 From Gaussian Elimination to Cholesky	24
3.3 DenseSolve Kernel Architecture	24
3.3.1 Two-Stage DATAFLOW Pipeline	25
3.3.2 Interface Design	25
3.3.3 DenseSolve Design Space Parameters	25
3.4 SparseSolve Kernel Architecture	26
3.4.1 Why a Separate Sparse Kernel	26
3.4.2 Pipeline Stages: K-Formation $\rightarrow$ Factorization $\rightarrow$ Triangular Solves	27
3.4.3 Memory Layout: CSC for K , CSR for L , On-Chip vs DDR Placement	27
3.4.4 Interface and DDR Streaming	27

4	Implementation	29
4.1	DenseSolve Kernel Implementation	29
4.1.1	K-Formation Module	29
4.1.2	LDL ^T Solver Module	29
4.1.3	DATAFLOW Pipelining	31
4.1.4	Host Interface	31
4.2	SparseSolve Kernel Implementation	31
4.2.1	Host-Side Preparation	32
4.2.2	Sparse K-Formation in CSC	32
4.2.3	Simplicial Up-Looking LDL^T Factorization	33
4.2.4	Sparse Forward / Back Substitution	34
4.2.5	URAM/BRAM Placement	35
4.2.6	Host Driver	35
4.3	Clock-Frequency Targets	35
4.4	IPM Pipeline Software	35
4.4.1	Hardware Proxy Classes (CDenseSolveProxy, CSparseSolveProxy)	35
5	Evaluation	37
5.1	Experimental Setup, Metrics, and Methodology	37
5.1.1	Test Inputs	37
5.1.2	Latency Measurement	37
5.1.3	Energy Measurement	38
5.1.4	Numerical Accuracy	38
5.1.5	Speedup Metric	39
5.1.6	Resource Reports	39
5.2	Dense Kernel: Latency, Resources, Energy and Scaling Analysis	39
5.2.1	Latency, Resources and Scaling	39
5.2.2	Energy per Solve	43
5.2.3	Numerical Accuracy	44
5.2.4	Unrolling-Factor Design Space	45
5.3	Sparse Kernel: Latency, Resources, Energy and Scaling Analysis	48
5.3.1	Resource Footprint	48
5.3.2	Latency, Energy and Scaling	48
5.3.3	Numerical Accuracy	52
5.4	HW Kernel Selection: Dense-HW vs Sparse-HW Crossover	53
5.5	End-to-End IPM	55
5.6	FEN Urban-Energy Case Study	57
6	Conclusion	59
6.1	Summary of Contributions	59
6.2	Dissemination	59
6.3	Findings and Lessons Learned, Self-Assessment	60
6.4	Future Work	60

Bibliography 62

A Declaration on the use of generative AI 64

B Project Repositories 65

 B.1 GitHub 65

 B.2 Google Drive 65

C Project Description 66

List of Acronyms

Acronym	Expansion
ARM	Advanced RISC Machines
AXI4	Advanced eXtensible Interface
B&B	Branch-and-Bound
BRAM	Block RAM
CMA	Contiguous Memory Allocator
CPU	Central Processing Unit
CSC	Compressed Sparse Column
CSR	Compressed Sparse Row
DDR	Double Data Rate (memory)
DMA	Direct Memory Access
DSP	Digital Signal Processing (slice)
FEN	Research Center for Energy Networks (ETH Zürich)
FF	Flip-Flop
FIFO	First-In, First-Out
FLOP	Floating-Point Operation
FPGA	Field-Programmable Gate Array
HLS	High-Level Synthesis
II	Initiation Interval
ILA	Integrated Logic Analyzer
IPM	Interior-Point Method
KKT	Karush–Kuhn–Tucker conditions
LP	Linear Program
LUT	Look-Up Table
MILP	Mixed-Integer Linear Program
OI	Operational Intensity
PDHG	Primal–Dual Hybrid Gradient
PL	Programmable Logic
PMT	Power Measurement Toolkit (ASTRON)
PS	Processing System
RCM	Reverse Cuthill–McKee ordering
REHO	Renewable Energy Hub Optimizer
RTL	Register-Transfer Level
SNR	Signal-to-Noise Ratio
SoC	System-on-Chip
SPD	Symmetric Positive Definite
UC	Unit Commitment
URAM	UltraRAM

Chapter 1

Introduction

As cities grow and energy demands rise, urban planners increasingly use computational models to optimize energy systems at the campus, district, and even city scale. Tools like the Renewable Energy Hub Optimizer (REHO) [20] write these planning problems as Mixed-Integer Linear Programs (MILPs), which mix continuous and discrete decisions. Installation sizing (number of solar panels, heat-pump locations), storage options (batteries), and scheduling are optimized together under physical and economic constraints. These models are powerful but costly: solving large MILPs is computationally expensive, and the problem only gets harder as digital-twin models grow in size and complexity.

Computational bottleneck. Most MILP solvers have a two-level structure. A Branch-and-Bound algorithm explores a tree of subproblems, each built by fixing some integer variables and solving the resulting Linear Program (LP) relaxation. Modern LP solvers typically use an Interior-Point Method (IPM) [10], which refines a solution step by step by solving a sequence of linear systems. The main cost of each IPM iteration is a linear solve: factorizing and solving a system matrix. Both steps scale cubically with the number of constraints, and at the urban-energy-model scales this kernel dominates the total runtime.

Limitations of existing approaches. General-purpose solvers like HiGHS [18] and Gurobi run these workloads on CPUs, but their energy use and latency can become too high for large or real-time applications. FPGAs offer an attractive alternative: their fine-grained parallelism and custom datapaths can speed up regular, compute-bound kernels while using a fraction of a CPU’s energy. Earlier work has accelerated LP and MILP solving on FPGAs only in narrow settings: an LDL^T matrix-inversion kernel in isolation [19], the Simplex method beneath a Branch-and-Bound search [9], and mixed-integer quadratic programming for real-time robot control [23]. But these are designs for small, fixed-size problems that do not explore the scaling or design-space trade-offs that urban-scale MILP demands (Section 2.7.3).

Approach. This project accelerates MILP solving on FPGAs with a hardware-software co-design. The Branch-and-Bound layer stays on the CPU, while the costly linear-algebra kernels of the inner IPM run on High-Level Synthesis (HLS) accelerators in the programmable logic. A sparse version of the same factorization, in compressed sparse column (CSC) format, is also explored for constraint matrices too large for the dense kernel. Each kernel is parameterized by unrolling factor, memory placement, and fixed-point precision, and this design space is swept on a Xilinx ZCU104 to find Pareto-optimal trade-offs between latency, energy, and on-chip resources. The goal is not to replace general-purpose solvers, but to show that targeted acceleration of the kernels at the bottom of the stack delivers meaningful gains.

Why FPGA acceleration. Energy and latency are the main drivers. Urban-energy models are re-solved across many scenarios and sensitivity studies, so even a small reduction in energy-per-

solve or runtime is worthwhile over a full planning campaign—and a tool meant to optimize energy systems should itself run efficiently. A custom accelerator also removes the dependence on proprietary, license-gated solvers: the main kernel runs on an open hardware-software pipeline that can be deployed and reproduced without commercial software. Thread-level parallelism inside a Branch-and-Bound driver also scales poorly. Even the commercial Gurobi solver reports only a $2.94\times$ speed-up at 12 threads on the standard MILP benchmark (Section 2.7). Spatial parallelism inside a single LP iteration, the kind FPGA dataflow pipelines provide, is therefore a complement rather than a competitor.

Central claim. Targeted FPGA acceleration of the dense and sparse linear-algebra kernels at the core of an Interior-Point LP solver can deliver significant speedup and energy reduction over a CPU baseline for MILP workloads.

Project objectives. The project is framed around four obligatory objectives set by the project description:

1. Profile an IPM-based MILP solver and identify the kernels that dominate its runtime;
2. Implement and optimize at least two of these kernels on the FPGA through High-Level Synthesis;
3. Explore the design space of each kernel and report at least three Pareto-optimal operating points trading off energy, latency, and resource use;
4. Evaluate the accelerators on hardware, comparing energy, execution time, and solution accuracy against a CPU baseline.

Two additional objectives extend this scope: integrating the kernels into a complete MILP solver pipeline, and analysing the accuracy of the accelerated solutions against software solvers on urban-energy case studies.

Results overview. Both kernels were synthesized and measured on a Xilinx ZCU104. The dense $LDL^\top$ kernel is the clearest gain: at $n = m = 256$ it solves the normal-equations system $34\times$ faster than the ARM Gaussian baseline— $14.3\times$ against the same algorithm in software—for $6.2\times$ less energy, and holds its signal-to-noise ratio above 100 dB across the whole size sweep. The sparse kernel is more situational: it overtakes the CPU on banded and moderately dense systems (up to $3.7\times$ faster for $3.6\times$ less energy) but falls behind on the very sparse Unit-Commitment instances, where the short factor columns starve its pipeline. Chapter 5 reports these results in full.

Contributions. This project makes four contributions. First, it profiles an IPM-based MILP solver and locates the dominant cost at the targeted scales in the matrix factorization inside each interior-point iteration. Second, it implements and optimizes two HLS factorization kernels on the ZCU104: a dense $LDL^\top$ kernel that forms and factorizes the normal-equations matrix entirely

in on-chip UltraRAM (URAM), and a sparse counterpart that uses host-side symbolic analysis and CSC/CSR storage (compressed sparse column/row) to reach larger constraint matrices. Third, it explores the design space of each kernel over different parameters, producing quantified Pareto frontiers by evaluating it end to end against a CPU baseline on latency, energy, accuracy, and resource use to do a quantitative analysis. Finally, it profiles two urban-energy case studies (from the FEN at ETH Zürich [15]) to understand their structure and test one of them on the accelerator.

Project execution. The work was carried out over the fifteen-week spring semester (February–June 2026), organized into four work phases followed by a final reporting phase, as shown in Figure 1.1. The first phase covered background study, codebase analysis, and CPU profiling. The second built the normal-equations matrix and the dense solver on the FPGA; the third extended the approach to a sparse solver. The fourth phase carried out the experimental evaluation and design-space exploration, with report writing running alongside the final experiments. The plan was followed as shown. The kernels were integrated into the inner interior-point LP solver and evaluated end to end (Section 5.5); integrating that solver into the outer Branch-and-Bound layer to form a complete MILP pipeline—one of the additional objectives—was set aside, in agreement with the supervisors, to concentrate on the quantitative evaluation.

Task / Deliverable	W1 Feb 16	W2 Feb 23	W3 Mar 02	W4 Mar 09	W5 Mar 16	W6 Mar 23	W7 Mar 30	BREAK	W8 Apr 13	W9 Apr 20	W10 Apr 27	W11 May 04	W12 May 11	W13 May 18	W14 May 25	W15 June 1
Phase 1: Understanding & Profiling																
Literature review (LP, MILP, IPM)																
Codebase analysis, Block diagram, CPU Profiling																
Plannification																
Phase 2: K building and dense solving on FPGA																
Environment familiarization and setup of first complete fonctionnal pipeline																
HLS: basic implementation																
HLS: evolution to a new methods (Conjugate Gradient, Cholesky ?)																
HLS: optimization for ressources, energy and latency																
Phase 3: Sparse solving on FPGA																
Exploration of sparse solving possiblity on FPGA																
HLS: basic implementation																
HLS: optimization for ressources, energy and latency																
Phase 4: Evaluation & Quantitative Analysis																
Design space exploration: vary parallelism, precision																
Pareto analysis: energy vs latency vs resources																
Compare FPGA vs CPU: energy, execution time, accuracy, (quantitative analysis)																
Phase 5: Report																
Final experiments & data collection																
Report writing																

Figure 1.1: Project execution over the fifteen-week spring semester (February–June 2026): four work phases—understanding and profiling, dense solving, sparse solving, and quantitative evaluation—followed by final reporting.

Chapter 2

Background and Related Work

This chapter covers the mathematical and technical background needed to understand the accelerator design: the optimization problem structure, the solver algorithm, the factorization method chosen for hardware, the fixed-point number format, and the target FPGA platform. It closes by reviewing related work linked to this project.

2.1 Mixed-Integer Linear Programming

A Mixed-Integer Linear Program (MILP) minimizes a linear cost under linear constraints, with some variables forced to integer values [1]:

$$\begin{aligned} \min_x \quad & c^\top x \\ \text{s.t.} \quad & Ax = b, \quad x \geq 0, \quad x_j \in \mathbb{Z} \quad \forall j \in \mathcal{J}. \end{aligned} \tag{2.1}$$

Here $x \in \mathbb{R}^n$ is the decision vector, $A \in \mathbb{R}^{m \times n}$ the constraint matrix, $b \in \mathbb{R}^m$ the right-hand side vector of the constraints, $c \in \mathbb{R}^n$ the cost vector, and $\mathcal{J} \subseteq \{1, \dots, n\}$ the set of integer variables.

MILPs are usually solved with the Branch-and-Bound algorithm (B&B) [1], which explores a binary search tree. At each node, the integer requirements are dropped to get a Linear Program (LP)—a continuous problem that is much easier to solve—whose solution gives a lower bound on the optimal cost. If that solution is already integer-valued, the node yields a candidate answer. Otherwise, an integer variable x_j with a fractional value $\bar{x}_j$ is picked, and two child nodes are created by adding the constraints $x_j \leq \lfloor \bar{x}_j \rfloor$ and $x_j \geq \lceil \bar{x}_j \rceil$ (for a binary variable, this just fixes x_j to 0 or 1). A subtree is pruned as soon as its LP bound is worse than the best known integer solution:

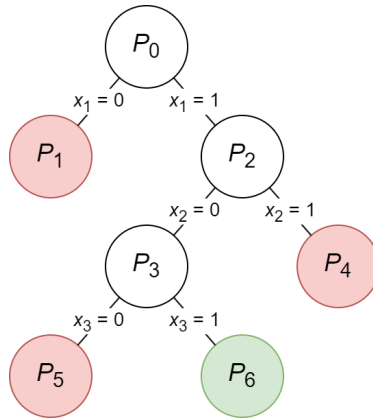


Figure 2.1: Branch-and-Bound search tree. Green: integer-feasible leaf. Red: pruned subtrees.

The cost of B&B is dominated by the LPs solved at each node: a single MILP can need hundreds or thousands of nodes, each running an LP for 10–20 iterations.

2.2 Interior-Point Method for Linear Programming

The LP at each B&B node is solved with a primal-dual Interior-Point Method (IPM) [10]. The algorithm maintains a triple (x, y, s) with $x > 0$ and s (dual slack vector with one value per non-negativity constraint) > 0 throughout: this strict positivity is what makes the method “interior”. Each iteration applies one Newton step toward the LP’s Karush–Kuhn–Tucker (KKT) optimality conditions.

For the LP $\min c^\top x$ subject to $Ax = b, x \geq 0$, the KKT conditions involve two auxiliary vectors: $y \in \mathbb{R}^m$ (one per equality constraint) and $s \in \mathbb{R}^n$ (one per non-negativity constraint). A triple (x, y, s) with $x, s \geq 0$ is optimal if and only if:

$$\begin{aligned} Ax &= b, & \text{(primal feasibility)} \\ A^\top y + s &= c, & \text{(dual feasibility)} \\ x_i s_i &= 0 \quad \forall i. & \text{(complementarity)} \end{aligned} \tag{2.2}$$

The first two conditions are linear; the third, complementarity, is not, and it forces $x_i = 0$ or $s_i = 0$ for every i . The IPM replaces it with the smoothed condition $x_i s_i = \sigma \mu$, where $\mu = x^\top s / n$ measures the current duality gap and $\sigma \in (0, 1)$ is a fixed centering parameter ($\sigma = 0.1$ here). Each iteration computes a Newton step from the smoothed system and shortens that step if needed to keep x and s strictly positive (safety factor $\alpha = 0.99$). As μ shrinks across iterations, the smoothed condition tightens automatically toward true complementarity. The algorithm stops when the primal residual $\|Ax - b\|_\infty$, the dual residual $\|A^\top y + s - c\|_\infty$, and the gap μ are all below a chosen tolerance.

The Newton step $(\Delta x, \Delta y, \Delta s)$ comes from linearising the smoothed KKT system around the current iterate. Eliminating Δx and Δs algebraically (a Schur complement) reduces it to a much smaller system in only m unknowns:

$$K \Delta y = r, \quad K = A \operatorname{diag}(D) A^\top, \tag{2.3}$$

where $D_j = x_j / s_j > 0$ is a diagonal scaling and r is built from the current residuals. This reduced form is called the normal equations. Solving it once per IPM iteration dominates the per-iteration cost (Section 3.1) and is the target of FPGA acceleration in this project.

Two properties of K shape the hardware design. First, even though A is sparse, K is much denser: K_{ij} is non-zero whenever rows i and j of A share any non-zero column, which happens for most row pairs at this density. Second, K is Symmetric Positive Definite (SPD): symmetric because D is diagonal, and positive definite because $D > 0$ holds throughout the IPM. SPD is what allows replacing general Gaussian elimination by the cheaper $LDL^\top$ factorisation (Section 2.3).

2.3 Direct Methods for Linear Systems

2.3.1 Gaussian Elimination with Partial Pivoting

For a general invertible K , the standard solver is Gaussian elimination with partial pivoting [10]. At step k (for $k = 0, \dots, m-1$), the row with the largest $|K_{ik}|$ among rows $i \geq k$ is swapped into row k —the *partial pivot*, used to avoid dividing by a small number—and every row below is updated to zero out column k below the diagonal. The right-hand side r is updated alongside. After m such steps, the system is upper triangular and is solved by back substitution. The cost is $\frac{2}{3}m^3$ FLOPs (floating-point operations) for the elimination plus m^2 for each back substitution.

2.3.2 $LDL^\top$ Cholesky Factorization

The $LDL^\top$ Cholesky factorization is the SPD-specific version: symmetry halves the arithmetic, and positive definiteness removes the need for pivoting. It decomposes [10]:

$$K = L D L^\top, \quad (2.4)$$

where L is unit lower-triangular (ones on the diagonal) and D is a positive diagonal matrix. The factorization proceeds column by column: for column j ,

$$D_j = K_{jj} - \sum_{k=0}^{j-1} L_{jk}^2 D_k, \quad (2.5)$$

$$L_{ij} = \frac{1}{D_j} \left(K_{ij} - \sum_{k=0}^{j-1} L_{ik} L_{jk} D_k \right), \quad i > j. \quad (2.6)$$

After factorization, the system $K \Delta y = r$ is solved in three steps:

1. **Forward substitution:** solve $Lz = r$ (unit lower-triangular, $O(m^2)$).
2. **Diagonal solve:** compute $w_i = z_i/D_i$ ($O(m)$).
3. **Back substitution:** solve $L^\top \Delta y = w$ (unit upper-triangular, $O(m^2)$).

The total cost is $O(m^3/3 + m^2)$ FLOPs, half that of Gaussian elimination. Table 2.1 compares the three main factorization approaches.

Table 2.1: Comparison of factorization methods for solving $K \Delta y = r$.

	Gaussian	$LL^\top$ Cholesky	$LDL^\top$ Cholesky
Applicability	Any invertible	SPD only	SPD only
FLOPs	$\frac{2}{3}m^3$	$\frac{1}{3}m^3$	$\frac{1}{3}m^3$
Pivoting required	Yes (partial)	No	No
Square root	No	Yes	No
Memory access	Irregular	Regular	Regular
Control flow	Data-dependent	Deterministic	Deterministic
FPGA suitability	Poor	Moderate	Excellent

The $LDL^\top$ variant is preferred over the standard $LL^\top$ Cholesky because it avoids square root operations, which have long latency and are difficult to pipeline in hardware [19]. Both share the same FLOP count and require no pivoting when the input is SPD.

2.4 Sparse Linear Algebra

The dense kernel of Section 4.1 fits the ZCU104 only while K is small enough to stay in URAM. At larger sizes, m reaches several thousand constraints and a dense K no longer fits the URAM+BRAM budget of Table 2.2. The constraint matrix A of REHO-style models is very sparse, so at these scales a sparse path is used that stores only the non-zeros of K and its factor L .

2.4.1 Sparse Storage Formats: CSC and CSR

A sparse matrix is stored compactly as three arrays: the non-zero values, and two index arrays that record where each value sits. Two layouts are used here. *CSC* (Compressed Sparse Column) orders the non-zeros column by column, so reading an entire column is one contiguous scan; *CSR* (Compressed Sparse Row) does the same row by row. The sparse kernel keeps K in CSC. It keeps the factor L in CSC, because the factorization produces L one column at a time, and additionally mirrors it in CSR, because the up-looking algorithm of Section 2.4.4 reads L row by row.

2.4.2 Fill-In and Symbolic Factorization

Factorizing a sparse matrix K creates new non-zeros in L at positions where K was zero. These extra entries, called *fill-in*, enlarge L and slow the factorization, so a sparse solver tries to keep them to a minimum.

The useful part is that *where* the non-zeros of L land depends only on the pattern of K , not

on its values. A *symbolic factorization* therefore computes those positions once, ahead of time, by walking an *elimination tree* that records which column affects which. Here the symbolic step runs once per problem on the host CPU with the algorithm of Davis [14]; its result is uploaded to the FPGA and reused across all IPM iterations, since only the diagonal D and the right-hand side change from one iteration to the next.

2.4.3 Fill-Reducing Ordering (RCM)

Reordering the rows and columns of K symmetrically changes how much fill-in appears, but leaves the solution unchanged. This project uses *Reverse Cuthill–McKee* (RCM) [13], a fast breadth-first graph traversal that clusters the non-zeros near the diagonal and so reduces fill. It was chosen for its simplicity. Because it affects only the amount of fill-in and not the numerical result, it acts as a tuning knob rather than a correctness requirement.

2.4.4 Simplicial Up-Looking Sparse $LDL^\top$

The sparse factorization follows the same column-by-column scheme as the dense one (Section 2.3), but touches only the non-zeros predicted by the symbolic step. The HLS kernel implements the *up-looking* variant of Davis [14]: each column j is built in three steps—gather column j of K , apply the updates from the earlier columns that touch it, then divide by the new diagonal entry D_j and store the finished column. As in the dense kernel, D_j is clamped to 1 when it turns non-positive, to avoid a division by zero under fixed-point rounding.

2.5 Fixed-Point Arithmetic

Fixed-point representation maps real values to integers by scaling with a fixed power of two. For example, the Q8.23 format uses 32 bits in total—1 sign bit, 8 integer bits, and 23 fractional bits—and encodes a real value as the stored integer divided by 2^{23} .

Range and precision. The representable range is $[-256, +256)$ with a resolution of $2^{-23} \approx 1.19 \times 10^{-7}$, providing roughly 7 decimal digits of precision.

Multiplication. Two Q8.23 values are multiplied by widening to 64 bits, computing the integer product, and right-shifting by 23 bits to restore the fixed-point format:

$$\text{FXP_Mult}(a, b) = (\text{int64}(a) \times \text{int64}(b)) \gg 23. \quad (2.7)$$

Division. The numerator is left-shifted by 23 bits before integer division:

$$\text{FXP_Div}(a, b) = (\text{int64}(a) \ll 23) / \text{int64}(b). \quad (2.8)$$

Fixed-point arithmetic avoids the complexity of floating-point hardware (exponent alignment, rounding modes, special values), resulting in simpler and more area-efficient circuits. The 32-bit multiply maps directly to the DSP48E2 slices on Xilinx UltraScale+ devices, which provide native 27×18 -bit multipliers [2].

2.6 FPGA Platform and High-Level Synthesis

2.6.1 FPGA Fundamentals

A Field-Programmable Gate Array (FPGA) is a reconfigurable silicon fabric in which logic blocks, memory blocks, and arithmetic primitives are wired together by programmable interconnect. Modern Xilinx UltraScale+ devices expose four resource types relevant to this work: LUTs (look-up tables) that implement arbitrary Boolean functions, flip-flops (FFs) that hold pipeline state, DSP48E2 slices (digital signal processing) that perform a fused multiply-accumulate in a single clock, and block RAM (BRAM)/URAM blocks that act as small, configurable on-chip SRAMs. A CPU time-multiplexes a fixed pipeline across many instructions; an FPGA instead lays out one custom datapath per kernel in space. Loops can be unrolled into parallel hardware lanes, and intermediate values flow directly between stages through registers and FIFOs instead of a cache hierarchy.

2.6.2 Xilinx ZCU104

The target device is the Xilinx ZCU104 evaluation board, featuring a Zynq UltraScale+ XCZU7EV system-on-chip [4]. This SoC integrates an FPGA programmable logic (PL) fabric with a quad-core ARM Cortex-A53 processing system (PS), connected through AXI4 (Advanced eXtensible Interface) interfaces. Table 2.2 summarizes the available FPGA resources.

Table 2.2: XCZU7EV programmable logic resources.

Resource	Count	Notes
LUTs	230,400	Configurable logic
Flip-Flops	460,800	Registers
Block RAM	312 ($\times$ 36 Kib)	11.50 Mb total = 1.44 MB total
UltraRAM	96 ($\times$ 288 Kib)	Native 4K $\times$ 72-bit; 28.31 Mb total = 3.54 MB total
DSP48E2	1,728	27×18 -bit multiplier + 48-bit accumulator
On-chip memory	4.98 MB (BRAM + URAM combined)	

2.6.3 High-Level Synthesis

The hardware accelerators are designed using Vitis High-Level Synthesis (HLS) [3], which compiles annotated C++ code into register-transfer level (RTL) hardware. HLS pragmas control pipelining, loop unrolling, array partitioning, and memory binding, enabling rapid design space exploration without manual RTL coding.

2.7 Related Work

2.7.1 General-Purpose CPU MILP/LP Solvers

The state of the art for solving MILPs on CPUs is represented by two systems. HiGHS [18] is an open-source LP/MILP solver that combines dual simplex, an interior-point method with crossover, and an MILP layer with presolve and primal heuristics. Gurobi [17] is the commercial reference, with a more extensive presolve, additional heuristics, and proprietary cut-generation strategies. Both solvers operate on desktop or server CPUs and represent the practical baseline for general-purpose MILP work, including for the REHO energy models that motivate this project. The FPGA accelerator presented here is not a replacement for HiGHS or Gurobi: it does not implement presolve, branching heuristics, or cut generation, and at the urban-energy scale it is the per-iteration LP solve that is offloaded, not the entire MILP pipeline. A realistic deployment would still use a CPU solver as the outer driver, with the FPGA kernels invoked as a backend for the inner linear-system solves.

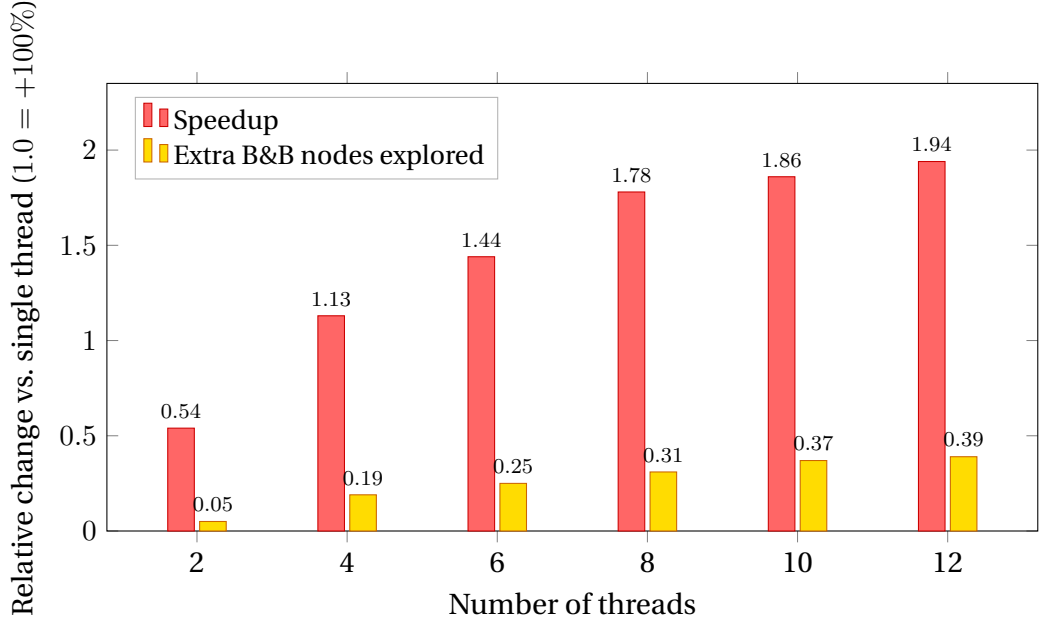


Figure 2.2: Parallel MIP scaling of a state-of-the-art Branch-and-Bound solver: speedup factor (0.54 means 54% faster than the single-threaded version) and growth in the number of explored nodes versus thread count. Reproduced from [16].

Figure 2.2 illustrates a known limit of Branch-and-Bound parallelism on CPUs: each extra thread expands the search tree faster than it shortens it. Going from one to twelve threads delivers only a 194% speed-up ($2.94\times$ the single-thread speed, far from the $12\times$ linear ideal) while exploring 39% more nodes, because concurrent workers redundantly explore subtrees that a serial driver would have pruned with a tighter incumbent bound. This caps the wall-clock benefit of multi-core hardware for MILP at well below the linear ideal. The FPGA acceleration pursued in this project is orthogonal: it leaves the B&B layer single-threaded on the ARM cores and instead exploits parallelism inside each LP-IPM iteration, where the factorisation and triangular solves are deterministic dataflow kernels that do not suffer from this redundancy effect.

The scale gap between this work and these CPU solvers is large. Real energy planning instances reach millions of constraints; their dimensions, sparsity, and a phase-by-phase Gurobi profile are quantified in Section 5.6. The FPGA kernels of this project accelerate only the inner factor-and-solve of a single barrier iteration, and only at sizes that fit on chip (at most a few thousand constraints).

2.7.2 Decomposition Methods for Energy-System MILP

Jacobson, Pecci, Sepulveda, Xu and Jenkins [21] tackled very large energy-planning MILPs with Benders decomposition. The idea is to split one big problem into two parts: the investment decisions (what to build) and the operational decisions (how to run it over time). Once the investment part is fixed, the operational part breaks into many small subproblems that can

be solved in parallel, letting the method handle far larger problems. This works at a different level from the present project. Benders makes the problem smaller before solving it, whereas the FPGA kernels here make each inner LP solve faster once it is being solved. The two can be combined: a Benders driver on the CPU could hand each small subproblem’s LP solves to an FPGA back-end like the one built here.

2.7.3 FPGA Acceleration of LP and MILP Solvers

Ingemarsson and Gustafsson [19] reported a hardware architecture for positive-definite matrix inversion based on the LDL^T decomposition and back-substitution. The work focuses on the factorisation kernel in isolation, but not embedded in a complete IPM pipeline and the matrices they target are very small (4x4 and 8x8). Like them, the present kernels divide by the pivot with a multiplicative Newton–Raphson reciprocal rather than a hardware divider (Section 4.2.3).

FPGA acceleration of *integer* programming has been little explored. Bayliss, Bouganis, Constantinides and Luk [9] implemented the Simplex method on a Virtex-4 FPGA and framed it explicitly as the engine that solves the LP relaxations inside an MILP Branch-and-Bound search, reporting roughly up to twenty times greater speed-up over a CPU LP solver. That work shares the division of labour used here—repeated LP relaxations accelerated in hardware beneath a Branch-and-Bound driver—but solves each relaxation with Simplex rather than an interior-point method.

The mixed-integer *quadratic* programming (MIQP) solver of Shimai, Tani, Noguchi, Kawaguchi and Yoshimoto [23] runs Branch-and-Bound on the FPGA and solves a relaxation at each node of a binary search tree for real-time mobile-robot control. They achieved significant gains with pipelined architectures and parallel processing. The relaxation there is a quadratic program rather than the linear program of this project, and the problem dimensions are fixed in the datapath at synthesis time, so the design is not directly comparable to the more general and variable-size MILP solver here.

Chapter 3

Design

This chapter presents the design decisions behind the dense and sparse FPGA accelerators.

Figure 3.1 shows how the accelerator is split between the ARM Cortex-A53 processing system (PS) and the programmable logic (PL). The PS handles the MILP layer (Branch-and-Bound, LP-relaxation construction) and the IPM control flow (residual computation, μ check, fraction-to-boundary step, direction recovery, variable update). The PL hosts only the K-building kernel and the factorization-and-solving kernel: given the current A , the diagonal scaling D , and a right-hand side r , it forms $K = A \text{diag}(D) A^\top$, factors it via $LDL^\top$, and returns Δy . The profiling of Section 3.1 justifies this narrow split: K formation and the dense solve together account for over 95% of per-iteration FLOPs, while the rest are low-cost vector operations or control logic that would add host-FPGA I/O without a matching speedup. An initial plan considered moving the full IPM iteration body to the FPGA, but profiling led to this narrower scope.

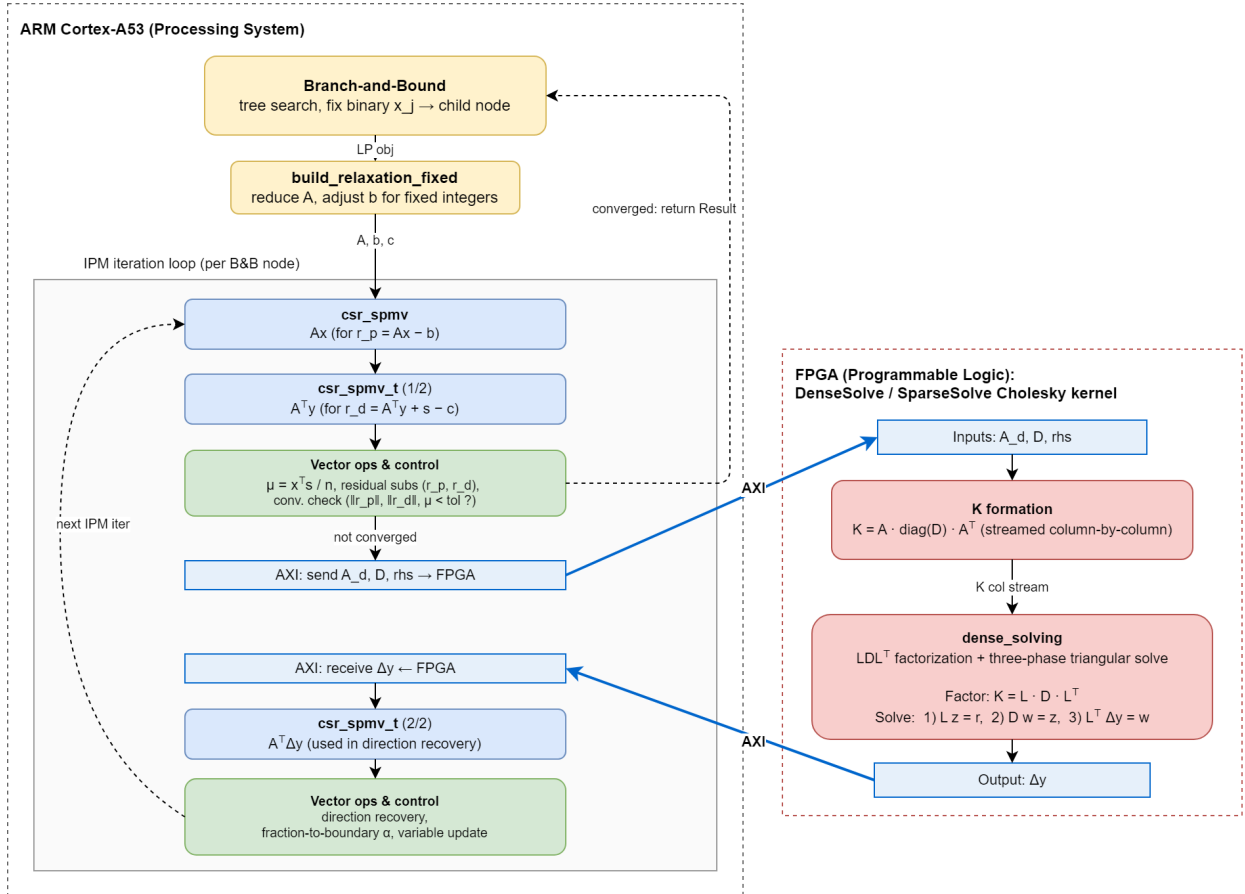


Figure 3.1: System architecture of the FPGA-accelerated LP-IPM solver: the PS part (left) runs the IPM and MILP control flow, and the PL part (right) hosts K building and factorization+solving kernels.

3.1 Computational Bottleneck and Roofline Analysis

The IPM solver calls four main kernels per iteration (setting aside the vector operations, which are negligible at scale):

- **csr_spmv.** A sparse matrix-vector product over a matrix with $\text{nnz} = m \cdot n \cdot d$ (d = density) non-zeros costs $2 \cdot \text{nnz}$ FLOPs (one multiply and one add per non-zero).
- **csr_spmv_t ($\times 2$).** The transposed product $A^\top v$ has the same non-zero structure and thus the same per-call cost. It is called twice per IPM iteration, doubling the count.
- **K formation.** Computing $K = A \text{diag}(D) A^\top$ requires, for each of the m^2 entries $K_{ij} = \sum_k A_{ik} D_k A_{jk}$, one multiply ($A_{ik} \cdot D_k$), one multiply (result $\cdot A_{jk}$), and one accumulation: 3 FLOPs per (i, j, k) triple. The total cost is therefore $3 \cdot m^2 \cdot n$ FLOPs.
- **dense_solving.** The CPU baseline uses Gaussian elimination with partial pivoting, which costs $\frac{2}{3}m^3$ FLOPs (Section 2.3). The FPGA kernel replaces this with $LDL^\top$ factorization at $\frac{1}{3}m^3$ (Section 3.2).
- **Vector operations and others.** The remaining kernels are vector updates and reductions (e.g., computing the residuals, the duality measure μ , and the fraction-to-boundary step). These are $O(m)$ or $O(n)$ operations and contribute negligibly to the total FLOP count at scale.

K formation ($O(m^2n)$) and the dense solve ($O(m^3/3)$) together account for nearly all FLOPs per iteration; the remaining kernels are negligible.

To confirm this on measured runtime, the CPU baseline was profiled on the ZCU104 processing system at $m = n = 256$ and density 0.10 (Table 3.1): K formation accounts for 63.2% of the solve time and the dense solve for 36.4% (99.6% combined), while the sparse products and vector operations stay below 0.5% together.

Table 3.1: Measured CPU-baseline runtime breakdown of an IPM solve at $m = n = 256$, density 0.10 ($\text{nnz} = 6585$, 10 IPM iterations, total 1478.2 ms), profiled on the ZCU104 Cortex-A53.

Stage	Time (ms)	Share
csr_spmv	0.57	0.0%
csr_spmv_t ($\times 2$)	1.70	0.1%
K formation	933.77	63.2%
dense_solving	538.45	36.4%
Vector operations and others	3.40	0.2%
Total	1478.23	100%

The dense solve and K-formation are therefore the acceleration targets.

The roofline model [24] classifies kernels as memory-bound or compute-bound based on their Operational Intensity (OI): the ratio of arithmetic operations to bytes of data transferred.

For the dense solve, the $m \times m$ matrix K fits in on-chip memory at the 72-bit TFXP width (9 B per element in URAM, matching the URAM block's native 72-bit word, so each element fills one URAM word with no padding). The operational intensity is then:

$$\text{OI}_{\text{dense_solve}} = \frac{(1/3)m^3}{9m^2} = \frac{m}{27} \text{ FLOP/byte.} \quad (3.1)$$

For K formation, the FPGA kernel buffers A on-chip and streams K column-by-column to the solver under DATAFLOW, so the off-chip traffic is dominated by the one-shot read of A (mn elements at 9 B per element, $9mn$ bytes total). With the $3m^2n$ FLOPs of the K-formation bullet above, the operational intensity is:

$$\text{OI}_{\text{K_form}} = \frac{3m^2n}{9mn} = \frac{m}{3} \text{ FLOP/byte.} \quad (3.2)$$

The ZCU104 DDR4 interface has a peak bandwidth of 19.2 GB/s (64-bit channel at 2400 MT/s) [4]. The ARM Cortex-A53 quad-core PS, four cores at 1.5 GHz [5] with one NEON FP64 FMA per cycle per core [7], reaches a peak FP64 throughput of $4 \times 2 \times 1.5 = 12$ GFLOP/s (giga-FLOP per second); against the 19.2 GB/s peak DDR4 bandwidth this places the CPU-side roofline ridge at ~ 0.63 FLOP/byte.

K formation's OI of $m/3$ exceeds this ridge for any $m \geq 2$, and the dense solve's OI of $m/27$ for any $m \geq 17$. Both kernels are therefore firmly compute-bound on the Cortex-A53 baseline across the problem sizes targeted in this project ($m \geq 100$, Table 3.2): wall-clock runtime is set by the 12 GFLOP/s peak FP64 throughput, not by the 19.2 GB/s DDR4 bandwidth (Figure 3.2). This justifies the focus on implementing these two kernels in the FPGA's programmable logic.

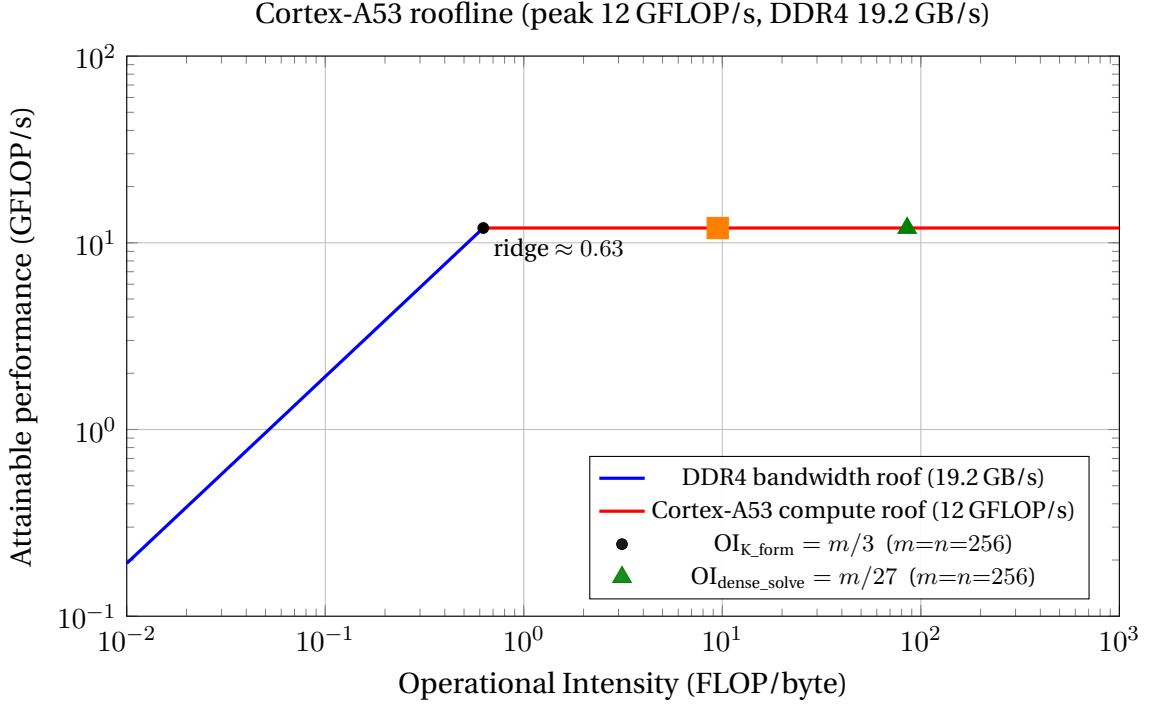


Figure 3.2: Roofline analysis against the ZCU104 ARM Cortex-A53 baseline.

The next step is to ensure that the FPGA implementation can achieve these compute-bound regimes by fitting all working data on-chip.

Table 3.2 shows that all benchmark sizes fit within the 4.98 MB on-chip memory budget. The table uses the chosen 72-bit TFXP width (9 B per element in URAM, matching the URAM block’s native 72-bit word).

Table 3.2: On-chip memory footprint at representative problem sizes.

Data structure	$m=n=128$	$m=n=256$	$m=n=512$
Ad matrix ($m \times n$, 9 B each)	147 KB	590 KB	2.36 MB
K matrix ($m \times m$, 9 B each)	147 KB	590 KB	2.36 MB
Working vectors ($\sim 5 \times m$, 9 B each)	5.8 KB	11.5 KB	23 KB
Total	300 KB	1.19 MB	4.74 MB

The $m = n = 512$ column shows that the total footprint (4.74 MB) is technically below the 4.98 MB on-chip budget. However, HLS requires each array to use a single storage type (BRAM or URAM, not split between them) [3], and the XCZU7EV provides only ~ 3.5 MB of URAM and ~ 1.4 MB of BRAM [5]. At 72-bit TFXP, neither of the two 2.36 MB matrices (Ad and K) fits in BRAM, so both must go to URAM; together they exceed the URAM capacity and synthesis fails. The maximum deployed kernel reaches $m = n = 384$ (Section 5.2.4).

3.2 From Gaussian Elimination to Cholesky

The software solver on the CPU uses Gaussian elimination with partial pivoting to solve the normal equations system $K \Delta y = r$. This is the standard approach for general-purpose solvers: it works for any invertible matrix and provides good numerical stability through row pivoting. However, Gaussian elimination is a poor fit for FPGA acceleration for two reasons.

First, partial pivoting introduces data-dependent control flow. At each elimination step, the algorithm searches a column for the largest absolute value, then conditionally swaps two rows. The pivot selection and row swap produce irregular memory access patterns and data-dependent branching that prevent deterministic pipelining.

Second, the algorithm performs $2m^3/3$ floating-point operations, and its irregular access pattern makes it difficult to exploit parallelism: the row swap destination is not known until the pivot search completes, stalling any pipelined execution.

The key observation is that the matrix $K = A \text{diag}(D) A^\top$ is *symmetric positive definite* by construction (Section 2.2). This symmetric positive definite guarantee unlocks the $LDL^\top$ factorization (Section 2.3), which eliminates both problems:

1. **Deterministic control flow.** The SPD property guarantees that all diagonal elements remain positive during factorization, so no pivoting or row swaps are needed. Every memory access and every branch is statically predictable, enabling fully pipelined execution on FPGA.
2. **Half the arithmetic cost.** The factorization requires only $m^3/3$ operations instead of $2m^3/3$, directly halving the computation for the most expensive kernel.
3. **No square roots.** Unlike the standard $LL^\top$ Cholesky, the $LDL^\top$ variant avoids square root operations entirely, replacing them with a single division per column.

Both the Gaussian and $LDL^\top$ paths are in the HLS codebase, selectable via a compile-time flag. Early benchmarking confirmed the advantage: at $m = 256$, switching from Gaussian elimination to $LDL^\top$ improved the FPGA speedup at the same unrolling factor ($U = 8$, Section 5.2).

3.3 DenseSolve Kernel Architecture

The first version of DenseSolve was built around Gaussian elimination with partial pivoting (based on the provided software Gaussian baseline). The solver worked in place on the full matrix K in URAM. Because partial pivoting needs global visibility of the column being eliminated, K had to be fully resident before the solve could start: K formation and the factorization therefore ran strictly back to back, with no overlap (no dataflow). This Gaussian path is still selectable through the compile-time `USE_CHOLESKY` switch in `dense_solve.cpp` and is kept as the comparison baseline in Chapter 5.

Two limitations of that first design motivated the move to Cholesky $LDL^\top$. First, the data-dependent pivot search and row swaps prevented deterministic pipelining of the solver loop. Second, the all-at-once dependency on K blocked any concurrent execution with K formation. The SPD structure of K (Section 3.2) removes both obstacles: pivoting becomes unnecessary, so the solver can consume one column of K at a time. The current `DenseSolve` kernel therefore fuses K formation and $LDL^\top$ factorization into a single HLS top-level function. The two phases communicate column by column through an HLS stream and run concurrently under a `DATAFLOW` pragma, eliminating the need to materialize K in full before the factorization starts. At $m = 256$, $U = 32$ (unrolling factor), this combined algorithmic and architectural change raised the FPGA speedup (Section 5.2).

3.3.1 Two-Stage DATAFLOW Pipeline

The kernel is split into two concurrent modules connected by an HLS stream (hardware FIFO):

1. **K-formation** (`k_formation_hw`): computes columns of the lower triangle of $K = A \text{diag}(D) A^\top$ one at a time, streaming each column into a FIFO as soon as it is ready.
2. **$LDL^\top$ solver** (`cholesky_solve_hw`): reads columns from the FIFO, performs the factorization incrementally (column by column), then executes the three-phase triangular solve.

The `DATAFLOW` pragma instructs the HLS tool to execute these two modules concurrently: while the K -formation module computes column $j + 1$, the solver processes column j .

3.3.2 Interface Design

The top-level `DenseSolve` kernel exposes:

- An **AXI4 master** port for burst DMA (direct memory access) transfers of input data (A_d , D , rhs) and output (Δy). Input data is packed contiguously: $A_d[m \times n] \mid D[n] \mid \text{rhs}[m]$.
- An **AXI4-Lite slave** port for control registers: input dimensions (m, n) , a `skipAd` flag, and the start/done handshake.

The `skipAd` flag allows the host to cache the A_d matrix across IPM iterations within a single LP solve. Since A does not change between iterations (only D and rhs are updated), this avoids redundant DMA transfers of the largest input array.

3.3.3 DenseSolve Design Space Parameters

Three parameters define the design space explored in this project:

Unrolling factor and array partitioning. Controls the number of parallel multiply-accumulate (MAC) operations per clock cycle in the inner loops of K formation and $LDL^\top$ factorization. Values between 1 and 32 are explored. The unrolling factor directly determines the number of DSP slices consumed and the degree of array partitioning required for conflict-free parallel memory access. Each array accessed in the unrolled inner loop must be cyclically partitioned with a factor matching the unrolling factor. This ensures that the U parallel iterations access U distinct memory banks, avoiding port conflicts. The K matrix is partitioned along its column dimension (dim 2), the accumulator arrays (`sum`, `partial_sum`) are fully partitioned.

K storage binding. The K matrix (up to $384 \times 384 \times 9$ bytes = 1327 KB) can be stored in either Block RAM or UltraRAM. URAM blocks are $8\times$ larger than BRAM blocks (288 Kib vs. 36 Kib), requiring fewer instances and reducing routing congestion. The `BIND_STORAGE` pragma directs HLS to use URAM with true dual-port access (`ram_t2p`).

Fixed-point precision (Q-format). All numerical data uses a 72-bit signed fixed-point type `TFXP`, split as $Q(int).(\textit{frac})$ with $1 + \textit{int} + \textit{frac} = 72$ bits. The integer/fractional split is exposed at build time via the `Q_INT_BITS` and `Q_FRAC_BITS` macros from the `shared/constants.h` file.

3.4 SparseSolve Kernel Architecture

The DenseSolve kernel of Section 3.3 is capped at $m = 384$ by keeping the dense K matrix on-chip, but this ceiling is too small for the target case studies (the UC instances in Chapter 5 reach several thousand constraints). A second top-level kernel, `SparseSolve`, is therefore added. It reuses DenseSolve’s three-phase factor-and-solve structure— K -formation, $LDL^\top$ factorization, two triangular solves—but works directly on the CSC/CSR-stored sparse K and L , raising the on-chip ceiling to $m = 8192$.

3.4.1 Why a Separate Sparse Kernel

A natural alternative to a new kernel would be to reuse DenseSolve by partitioning the sparse system into dense subblocks small enough to fit its $m = 384$ ceiling. Standard CPU sparse-Cholesky libraries (CHOLMOD [12] for example) do something similar: their *supernodal* factorization merges together columns of the Cholesky factor that share the same nonzero pattern into a single dense block [14].

REHO-style urban-twin models behave differently. Their constraint matrix A has a network-like, irregular sparsity pattern: consecutive columns of L rarely share a below-diagonal pattern and supernodes collapse to one or two columns. At that size the blocked path offers no reuse and only adds the cost of detecting and assembling supernodes, which is why partitioning into dense subblocks was discarded as a way to reuse DenseSolve.

`SparseSolve` therefore uses the simplicial up-looking factorization of Section 2.4.4, which

processes one column at a time. Columns flow through the HLS dataflow pipeline one by one.

3.4.2 Pipeline Stages: K-Formation → Factorization → Triangular Solves

The column-by-column data dependency that motivates DATAFLOW in DenseSolve (Section 3.3) carries over directly to the sparse case: column j of L depends only on columns $0..j-1$ of K , so K-formation, factorization, and the forward triangular solve can all operate concurrently on different columns. SparseSolve is therefore split into three HLS modules wired by HLS streams (`k_col_stream` from `kform` to `factor`, `l_col_stream` from `factor` to `forward-solve`) with a dataflow design. Back-substitution runs sequentially after the region because it walks $L^\top$ in reverse column order and requires the fully written L .

3.4.3 Memory Layout: CSC for K , CSR for L , On-Chip vs DDR Placement

CSC for K . The `kform` stage produces one column of K at a time and streams the values into the `factor` stage. CSC is the natural layout because each column's row indices are produced consecutively and consumed in the same order downstream. The `factor` stage reads the CSC pattern (`K_colptr`, `K_rowidx`) directly from URAM/BRAM, which was loaded once at LP setup.

Dual CSC + CSR for L . The `factor` stage writes each entry of L into both CSC and CSR buffers in URAM because the two layouts serve different consumers. The CSR copy is read by the `factor` stage itself: the up-looking $LDL^\top$ recurrence computes column j by walking row j of L to gather the rank-1 updates from previously-emitted columns $k < j$ (`L_rowptr`, `L_colidx_csr`, `L_values_csr`). Row-walks cost $O(\text{nnz of the row})$ in CSR but $O(\text{nnz}(L))$ in CSC, so CSR is what keeps the inner loop sparse. The CSC copy serves both triangular solves: the streamed forward solve reads each CSC column j in turn. The dual emit is cheap because `L_csr_pos`, the map from CSC position to CSR slot, is precomputed once at LP setup from the symbolic factorisation.

On-chip vs DDR placement. The current design caps the size at $m = 8192 / \text{nnz} = 65536$, with all buffers on-chip. Going larger would require DDR streaming. An earlier draft tried streaming L to DDR but dropped it, because the sparse kernel was already difficult to accelerate beyond the software baseline. The work therefore focuses on the on-chip design and leaves DDR streaming as a future extension.

3.4.4 Interface and DDR Streaming

Interface. A single AXI4 master bundle (`master1`) carries every host-to-kernel and kernel-to-host transfer; an AXI4-Lite slave block exposes the scalar control registers (`m`, `n`, `nnz_A`, `nnz_K`, `nnz_L`, `mode`, plus the three iteration-count readbacks `kform_iters`, `factor_iters`, `triangular_iters` used for performance profiling). The control plane is designed around a SETUP/SOLVE mode register so that the immutable inputs (A pattern and values, K and L

symbolic patterns) are sent once per LP and the per-IPM-iteration cost is reduced to writing only D and b . This is the same I/O-narrowing lever as the DenseSolve `skipAd` flag (Section 3.3), generalized to the extra sparse-pattern arrays.

DDR streaming (considered, not adopted). An earlier draft planned to DDR-stream the L factor over dedicated AXI master bundles, to push the reachable problem size above the on-chip $\text{nnz}(L)$ ceiling. It was abandoned once experiments showed the sparse kernel was not yet faster than the software baseline even at on-chip sizes: adding DDR round-trip latency to every column of L would only have eroded performance further. The priority became optimizing the on-chip sparse solver first, leaving DDR streaming of L as a future extension once the inner loop is fast enough to absorb the extra DRAM traffic.

Chapter 4

Implementation

The design rationale for both kernels was given in Chapter 3; this chapter covers how that design is realized and optimized in HLS.

4.1 DenseSolve Kernel Implementation

4.1.1 K-Formation Module

The `k_formation_hw` function builds the lower triangle of $K = A \text{diag}(D) A^\top$ one column at a time and streams each column to the solver. For a column j , it first scales row j of A by D and stores the result in `acj_dj`. That array is fully partitioned (`ARRAY_PARTITION complete`), so all of its entries can be read at once. The kernel then forms the dot products $\text{sum}[i] = \sum_k \text{acj_dj}[k] \cdot \text{Ad}[i][k]$ for every row $i \geq j$. The inner loop over k is unrolled by U . To feed it, `sum` and the rows of `Ad` are cyclically partitioned by U , so the U lanes reach separate memory banks without conflicts. Finally, the entries `sum[j..m-1]` are streamed out at `PIPELINE II=1` (initiation interval of one cycle).

4.1.2 LDL^T Solver Module

The `cholesky_solve_hw` function receives K columns from the HLS stream and performs the $LDL^\top$ factorization in-place, followed by the three-phase triangular solve.

Factorization

The factorization handles one column j at a time. Column j arrives from the stream into the local K array, which lives in URAM (`BIND_STORAGE type=ram_t2p impl=uram`) and is cyclically partitioned by factor U . The kernel first precomputes $v[k] = K[j][k] \cdot D_k$ for $k < j$, then the pivot $D_j = K[j][j] - \sum_{k < j} K[j][k] v[k]$. If fixed-point rounding makes D_j non-positive, it is clamped to 1 to avoid a later division by zero. Each off-diagonal factor is then:

$$L_{ij} = \frac{K[i][j] - \sum_{k < j} K[i][k] \cdot v[k]}{D_j}, \quad i > j.$$

The inner sum here is the performance-critical loop (Listing 4.1). With one accumulator, every multiply-subtract would chain through a single variable and block `II=1`. Instead, the products are spread across U fully-partitioned banks (`partial_sum[k % U]`), so each unrolled lane gets its

own accumulator. The dot product then runs at PIPELINE II=1 with UNROLL factor= U . A short loop sums the U banks, and the result overwrites the lower triangle of K in place.

```

1 // Off-diagonal: L[i][j] = (K[i][j] - sum(K[i][k]*v[k])) / Dv[j]
2 LOOP_CHOL_OFFDIAG: for (uint32_t i = j + 1; i < DENSE_MAX_M; i++) {
3     if (i >= m) break;
4
5     TFXP partial_sum[SOLVING_UNROLLING_FACTOR] = {0};
6     #pragma HLS ARRAY_PARTITION variable=partial_sum complete
7
8     LOOP_CHOL_DOT: for (uint32_t k = 0; k < DENSE_MAX_M; k++) {
9         #pragma HLS PIPELINE II=1
10        #pragma HLS UNROLL factor=SOLVING_UNROLLING_FACTOR
11        if (k >= j) break;
12        partial_sum[k % SOLVING_UNROLLING_FACTOR] -= FXP_Mult(K[i][k], v[k]);
13    }
14
15    TFXP sum_off = K[i][j];
16    LOOP_CHOL_REDUCE: for (uint32_t p = 0; p < SOLVING_UNROLLING_FACTOR; p++) {
17        #pragma HLS UNROLL factor=SOLVING_UNROLLING_FACTOR
18        sum_off += partial_sum[p];
19    }
20
21    K[i][j] = FXP_Mult(sum_off, inv_dj); // L[i][j] stored in-place
22 }

```

Listing 4.1: Partial-sum reduction in the off-diagonal factorization step (cholesky_solve.cpp). SOLVING_UNROLLING_FACTOR is the unroll factor U .

Triangular Solves

After factorization, the system is solved in three passes:

1. **Forward substitution** ($Lz = b$): sequential scan from $i = 0$ to $m-1$. Each element $z_i = b_i - \sum_{j < i} L_{ij} \cdot z_j$ has a loop-carried dependency on previously computed z values, limiting parallelism. The inner loop is pipelined at II=1. The inner dot product reuses the same partial-sum reduction over U accumulators as the off-diagonal factorization step, so the j -loop is also unrolled by U .
2. **Diagonal solve** ($Dw = z$): computes $w_i = z_i / D_i$ for all i , pipelined at II=1. Each iteration is independent.
3. **Back substitution** ($L^\top \Delta y = w$): reverse scan from $i = m-1$ down to 0, with the same loop-carried dependency structure as forward substitution. The $j > i$ inner dot product likewise uses the partial-sum reduction over U accumulators, then reduces them in a single unrolled tree.

The triangular solves are $O(m^2)$ and do not benefit from unrolling to the same degree as the $O(m^3)$ factorization, making them a diminishing fraction of total latency as m grows.

4.1.3 DATAFLOW Pipelining

The K-formation and $LDL^\top$ solver modules are wrapped in a DATAFLOW region:

```

1 #pragma HLS DATAFLOW
2 hls::stream<TFXP> k_col_stream;
3 #pragma HLS STREAM variable=k_col_stream depth=DENSE_MAX_M
4 k_formation_hw(m, n, Ad, D, k_col_stream);
5 cholesky_solve_hw(m, k_col_stream, b, dy);

```

Listing 4.2: DATAFLOW wrapper overlapping K-formation with Cholesky solve.

The stream acts as a hardware FIFO. As K-formation finishes column j , the solver starts reading it while column $j+1$ is still being built. This overlap works because column j of the $LDL^\top$ factorization depends only on earlier columns, never on later ones. This is one of the properties that motivated choosing $LDL^\top$ over Gaussian elimination.

4.1.4 Host Interface

On the ARM Cortex-A53, a driver class (CDenseSolveProxy) manages communication with the FPGA kernel:

1. **Memory allocation.** Physically contiguous buffers are allocated via the Linux CMA (Contiguous Memory Allocator) for DMA-capable access. The driver maintains a virtual-to-physical address mapping.
2. **Register configuration.** The AXI4-Lite control registers are written with the input/output physical addresses, the problem dimensions (m, n), and the skipAd flag (encoded in the upper bits of inputHeight).
3. **Kernel launch and synchronization.** The ap_start bit is set to launch the kernel, and the driver polls the ap_done bit for completion.
4. **Burst transfer configuration.** The AXI4 master port is configured with 32 outstanding read requests and a maximum burst length of 256 beats (the AXI4 maximum), balancing throughput against on-chip buffering.

4.2 SparseSolve Kernel Implementation

The top function SparseSolve runs in one of two modes. MODE_SETUP caches the immutable inputs (A in CSR plus the symbolic patterns of K and L) into on-chip URAM/BRAM and returns.

MODE_SOLVE is the per-iteration path: it chains the three stages of Sections 4.2.2–4.2.4, which run concurrently inside one DATAFLOW region connected by the two streams `k_col_stream` and `l_col_stream`. The back-substitution runs after the dataflow region, since it walks $L^\top$ in reverse column order and needs the fully written factor in URAM.

4.2.1 Host-Side Preparation

Before any kernel launch, a one-time host pass turns the constraint matrix A into the immutable arrays that MODE_SETUP uploads. The pass is purely symbolic: it uses only A 's sparsity pattern, not its values, so its result is reused across every IPM iteration and B&B node (Section 2.4.2). It runs once per problem, in seven steps:

1. **Format A .** Convert A to CSR with sorted column indices (Section 2.4.1).
2. **Order.** Compute a Reverse Cuthill–McKee permutation P of $AA^\top$ to pull non-zeros toward the diagonal (Section 2.4.3).
3. **Permute.** Reorder the rows of A by P .
4. **K pattern.** Keep the lower triangle of $AA^\top$, giving `K_colptr` and `K_rowidx`.
5. **Elimination tree.** Build the elimination tree of K (Section 2.4.2).
6. **Symbolic factorization.** Predict L 's fill pattern, giving `L_colptr` and `L_rowidx_csc` (Section 2.4.2).
7. **Dual-format maps.** Build the CSR view of L and the slot map `L_csr_pos`, so the factor stage can write each value into both layouts in $O(1)$ (Section 3.4.3).

The pass is implemented offline in Python: it extracts A from the Unit-Commitment model and serializes A with all symbolic arrays into a fixture. The driver loads that fixture once through MODE_SETUP (Section 3.4.4); after that, only D and b change per iteration. The pass stays on the host because it is irregular graph work whose result never changes between iterations.

4.2.2 Sparse K-Formation in CSC

The `sparse_kform_stream_hw` function computes the lower triangle of $K = A \text{diag}(D) A^\top$ in CSC order and streams each entry to the factor stage. Unlike the dense case, the output pattern (`K_colptr`, `K_rowidx`) is known a priori from the host-side symbolic analysis, so the kernel only fills in the values.

Step 1: Scatter row j of A with a stamp. For column j , the kernel scales the non-zeros of row j of A by D and scatters them into a dense buffer `adj_dense[k] = Ajk · Dk`. It also writes j into a parallel stamp array `adj_stamp[k]`. Both arrays sit in BRAM (`BIND_STORAGE type=ram_2p`) and the loop runs at `II=1`.

Step 2: Stamp-guarded sparse dot product. For each non-zero row i listed in column j , the kernel walks row i of A and adds $A_{ik} \cdot \text{adj_dense}[k]$, but only where $\text{adj_stamp}[k] = j$. The stamp marks which entries belong to the current column, so the dense buffer never has to be cleared between columns. The inner loop runs at $\text{II}=1$ with a two-way unroll.

Step 3: Stream output. Each K_{ij} is pushed to `k_col_stream`. Producer and consumer share the DATAFLOW region, so the factor stage starts on column j as soon as its first entry arrives, while K-formation moves on to column $j+1$.

4.2.3 Simplicial Up-Looking $LDL^\top$ Factorization

The `sparse_factor_stream_hw` function consumes `k_col_stream` and produces L (stored in both CSC and CSR layouts) plus the diagonal vectors D and D^{-1} . It implements the simplicial up-looking $LDL^\top$ algorithm of Section 2.4.4 column by column.

Working vector and mark array. A dense vector $w[0..m-1]$ in URAM accumulates the current column j . A second array `mark[0..m-1]` in BRAM stores, for each entry, the column that last wrote it. Comparing `mark[r]` with j then tells in one cycle whether $w[r]$ already holds a value for column j ; if not, the entry counts as zero. This first-touch scheme spares w a full zero-fill at the start of every column. Both arrays are cyclically partitioned by `SPARSE_UNROLL`.

Rank-1 updates with false-dependence pragmas. For each prior column k that feeds column j , the loop `UPDATE_COL_K` subtracts $L_{rk} \cdot D_k \cdot L_{jk}$ from $w[r]$ at every off-diagonal row r of column k , while a scalar accumulates the pivot D_j (Listing 4.3). This was flagged by looking at the Vitis HLS schedule report: this loop was missing its $\text{II}=1$ pipelining target. The report was blaming an inter-iteration dependency on w and `mark`: because the write index r is data-dependent, the tool cannot prove that two iterations address different entries and conservatively serialises them. The dependency is spurious, since r comes from the column's CSC row list, which the symbolic analysis guarantees to be strictly increasing, so no two iterations ever touch the same $w[r]$ or `mark[r]`. Declaring the four edges (RAW and WAW on each of w and `mark`) false restores $\text{II}=1$ pipelining, and simulation against the software reference returns identical output.

```

1 // u = D_k * L_jk ; the scalar diag accumulates the pivot D_j over all k
2 TFXP u = FXP_Mult(l_jk, Dv[k]);
3 diag -= FXP_Mult(l_jk, u);
4
5 // Subtract L_rk * D_k * L_jk from w[r] for each off-diagonal r of column k
6 UPDATE_COL_K: for (uint32_t p = cs_k; p < ce_k; p++) {
7 #pragma HLS PIPELINE II=1
8 #pragma HLS DEPENDENCE variable=w      type=inter direction=RAW false
9 #pragma HLS DEPENDENCE variable=w      type=inter direction=WAW false
10 #pragma HLS DEPENDENCE variable=mark type=inter direction=RAW false
11 #pragma HLS DEPENDENCE variable=mark type=inter direction=WAW false
12     uint32_t r = L_rowidx_csc[p];
13     if (r <= j) continue; // skip the diagonal block
14     TFXP delta = FXP_Mult(L_values_csc[p], u);

```

```

15 |   if (mark[r] != j) {           // first touch this column: initialise w[r]
16 |       w[r] = (TFXP)0 - delta;
17 |       mark[r] = j;
18 |   } else {                     // already live: accumulate into w[r]
19 |       w[r] = w[r] - delta;
20 |   }
21 | }

```

Listing 4.3: Rank-1 update of column j from one prior column k . The mark array flags first-touch so w never needs a per-column zero-fill, and the false DEPENDENCE pragmas keep the loop at II=1 (sparse_factor.cpp).

Pivot safeguard and dual-format emit. The accumulated scalar is the pivot D_j . As in the dense path, it is clamped to FXP_ONE when non-positive, and its reciprocal D_j^{-1} is cached for the back-solve. Each entry $L_{rj} = w[r] \cdot D_j^{-1}$ is then written to both the CSC and CSR layouts and pushed onto `l_col_stream`, so forward substitution can begin before the later columns finish.

Reciprocal via Newton–Raphson. The cached reciprocal D_j^{-1} avoids a fixed-point divide. An FPGA has no built-in divider, so one must be built from logic that yields the quotient one bit per cycle, with each bit waiting on the previous remainder. The kernel instead computes $1/D_j$ with FXP_Recip, which needs only fixed-point multiplies and so maps to DSP slices. It works in two stages. First, a fully-unrolled scan finds the highest set bit of D_j , which fixes its magnitude; one shift turns this into a power-of-two seed $x_0 \approx 1/D_j$ in the same Q-format. Then unrolled Newton steps $x \leftarrow x(2 - D_j x)$ refine the seed. At the Q-format’s fractional width this is done in about 32 cycles: some $2.7\times$ faster than the divider (about 85 cycles). These latencies were obtained by synthesising each operation in isolation as a standalone top function in Vitis HLS and reading the latency from the C-synthesis report. This method is a standard way to divide in fixed-point on FPGAs [22], and the closely related $LDL^\top$ hardware design of Ingemarsson and Gustafsson also inverts the diagonal in fixed-point [19] (section 2.7.3).

4.2.4 Sparse Forward / Back Substitution

Forward solve (streamed inside DATAFLOW). The forward solve $Lz = b$ runs inside the DATAFLOW region and consumes the factor column by column as it streams in, so it can start before the factorization finishes. For each column j it reads z_j , then LOOP_FWD_SCATTER subtracts $L_{rj} \cdot z_j$ from every entry z_r below it, reading L in CSC layout. The inner loop reaches II=1 with the same false-dependence technique as the rank-1 update (Listing 4.3), since the rows are again visited in increasing order.

Back solve (sequential, after DATAFLOW). The back solve runs on its own, after the dataflow region, because it walks $L^\top$ in reverse column order and needs the fully written L in URAM. It first scales the right-hand side by the inverse diagonal, then sweeps the columns from $m-1$ down to 0. For each column, LOOP_BACK_DOT accumulates into $U = \text{SPARSE_UNROLL}$ partial-sum accumulators (ARRAY_PARTITION complete), reusing the dense back-solve’s partial-sum

reduction so the inner loop reaches $II=1$.

4.2.5 URAM/BRAM Placement

The on-chip layout follows the plan of Section 3.4.3. The four wide value arrays (`A_values`, `L_values_csc`, `L_values_csr`, and the scratch w) go to UltraRAM, while the narrower index and pointer arrays go to BRAM. The short solver vectors (D , b , z , Δy) are read by all $U = 16$ lanes at once during the triangular solves, so they go to LUTRAM, cyclically partitioned by U .

4.2.6 Host Driver

The `CSparseSolveProxy` driver uses the two modes of Section 3.4.4. The first call of an LP solve runs `MODE_SETUP`, which packs the immutable inputs— A in CSR (converted to TFXP via `Double2Fxp`) plus the symbolic patterns of K and L —into one contiguous buffer that the kernel caches on chip. Every later iteration runs `MODE_SOLVE`, which rewrites only D and b , keeping the per-iteration transfer small.

4.3 Clock-Frequency Targets

The two kernels run at different PL clocks: `DenseSolve` at 200 MHz and `SparseSolve` at 150 MHz. Each value is a compromise: a higher clock favours throughput, but timing closure and resource use limit how high it can be pushed before place-and-route fails to meet timing.

4.4 IPM Pipeline Software

The Interior-Point Method (IPM) that drives the kernels is a standard primal-dual solver. Its floating-point implementation in `sw/src/lp_ipm.c` was contributed by the project’s supervisor (Denisa-Andreea Constantinescu). This project adds three pieces to the IPM software: the fixed-point variant of the iteration body; the two hardware wrappers `lp_ipm_hw.cpp` and `lp_ipm_sparse_hw.cpp` for the dense and sparse paths; and the proxy abstraction (Section 4.4.1) that lets one wrapper target either accelerator.

4.4.1 Hardware Proxy Classes (`CDenseSolveProxy`, `CSparseSolveProxy`)

Shared base. Both drivers inherit from `CAccelProxy` (`sw/drivers/`), reused from the hardware-software co-design course. It maps the kernel’s AXI4-Lite registers into user space and hands out physically-contiguous DMA buffers, recording each virtual→physical mapping so the driver can program the kernel’s address registers.

Per-kernel proxies. `CDenseSolveProxy` packs A once at LP-solve startup, then each iteration rewrites only D and b and sets `skipAd` (the high bit of `inputHeight`) so the kernel skips re-reading A . `CSparseSolveProxy` does the same with the `MODE_SETUP` / `MODE_SOLVE` discipline of Section 4.2.6. Both expose a single `Solve(D, b, dy)` call, which is what lets the IPM driver target either accelerator without changing its algorithm.

Chapter 5

Evaluation

This chapter reports a quantitative analysis of both HW kernels, DenseSolve and SparseSolve, swept across matrix size, sparsity, unrolling factor, and fixed-point precision, and compared against the ARM Cortex-A53 software baseline on latency, energy, FPGA resource cost, and numerical accuracy.

5.1 Experimental Setup, Metrics, and Methodology

All measurements in this chapter are taken on the same Xilinx ZCU104 board (Section 2.6.2). The board runs PetaLinux on the Zynq UltraScale+ XCZU7EV, and the FPGA bitstream is loaded before each test run. The CPU baseline is the on-board quad-core ARM Cortex-A53, not an external workstation. This keeps the comparison on the same SoC and matches the real question for an edge digital-twin workload: run the linear-algebra kernel on the embedded CPU, or offload it to the on-chip FPGA fabric?

5.1.1 Test Inputs

The dense and sparse benchmarks are generated by the testbench (`sw/src/denseSolveTestbench.cpp`, `sw/src/sparseSolveTestbench.cpp`). For each problem size, the constraint matrix A holds uniform random fixed-point values in $[-1, +1)$, the diagonal scaling D holds positive values between 0 and 1 (which keeps the normal-equations matrix symmetric positive definite), and the right-hand side b is uniform random in $[-1, +1)$. The random sequence is seeded with `srand(42)` at the start of each run. Every configuration — Gaussian vs Cholesky, software vs hardware, any unrolling factor, any Q-format — therefore sees the same input data, and the comparison is reproducible.

5.1.2 Latency Measurement

Wall-clock latency is measured with `clock_gettime(CLOCK_MONOTONIC_RAW)` at nanosecond resolution. Each test point is repeated until two conditions hold together: at least $N_{\min} = 20$ repetitions, to suppress noise, and at least $T_{\min} = 100$ ms of total compute time, so the energy measurement (Section 5.1.3) covers at least five PMT sampling windows. The reported latency is the mean per-call time over the N runs actually executed. The HW measurement covers everything the host sees: register setup, the `ap_start` pulse, polling `ap_done`, and the DMA transfer of the result back to user space; it excludes the one-time DMA buffer allocation and the bitstream load. The SW reference times only the compute phase. Input-format conversion (fixed-point $\rightarrow$

double and back) is timed separately (`elapsedTimeConv`) and reported alongside, not folded into the compute time.

5.1.3 Energy Measurement

Energy is measured with the ASTRON Power Measurement Toolkit (PMT) [8]. Three power rails are sampled at 50 Hz: CORE+LOGIC (`/sys/class/hwmon/hwmon0`), AUX (`hwmon1`), and BOARD (`hwmon2`). The BOARD rail covers the whole board and is the value reported by default.

The board-wide draw includes the always-on power of the processing system, the static leakage of the programmable logic, fans, and on-board peripherals. This static draw is large next to the dynamic energy of a single millisecond-scale kernel call, and would dominate the numbers if left in. To remove it, a one-time idle baseline is measured for 1 s with no workload at the start of each run, giving the at-rest power P_{idle} . For a window of duration T_{win} covering N calls, the dynamic (compute-only) energy per call is then

$$E_{\text{percall}} = \frac{E_{\text{total}} - P_{\text{idle}} \cdot T_{\text{win}}}{N}, \quad (5.1)$$

where E_{total} is the total energy accumulated by PMT over the window. All reported energy numbers in this chapter are therefore dynamic energy per call, not total board energy. PMT samples each rail's instantaneous power in watts (W) and integrates it over time to obtain energy in joules (J). Every per-call figure in this chapter is reported in millijoules (mJ).

The PMT samples at 50 Hz, so one window is $T_{\text{sample}} = 20$ ms wide. If T_{win} is close to T_{sample} , the dynamic energy is a small fraction of $P_{\text{idle}} \cdot T_{\text{win}}$, and the numerator of Equation 5.1 sits inside the noise of the idle-power estimate. The adaptive repetition scheme of Section 5.1.2 keeps the measurement above this noise floor: running to $T_{\text{min}} = 100$ ms (five PMT windows) and at least 20 repetitions makes T_{win} large enough for the dynamic energy to dominate the per-window noise. The per-call energy then divides by the repetitions actually run.

5.1.4 Numerical Accuracy

For every HW solve, two reference solutions are computed in double precision on the ARM core: a Gaussian-elimination reference and an LDL^T Cholesky reference (the same algorithm as the HW kernel, in doubles). The HW fixed-point output $\hat{x}$ is compared element-wise against both references. The per-test signal-to-noise ratio is

$$\text{SNR}_{\text{dB}} = 10 \log_{10} \frac{\sum_i x_i^2}{\sum_i (x_i - \hat{x}_i)^2}, \quad (5.2)$$

5.1.5 Speedup Metric

Throughout this chapter, "speedup" is the ratio of mean software latency to mean hardware latency on the same problem size, both measured on the ZCU104. Two variants are reported. The first compares HW against the Gaussian-elimination CPU reference, the conventional baseline for a general dense solve. The second compares HW against the $LDL^\top$ CPU reference, which runs the same algorithm as the HW kernel and so separates the hardware gain from the algorithm gain.

For the sparse kernel, only one ratio is reported: HW sparse $LDL^\top$ against the equivalent SW sparse $LDL^\top$. There is no Gaussian variant of the sparse path.

5.1.6 Resource Reports

FPGA resource counts (LUTs, FFs, DSP48E2 slices, BRAM, URAM) are taken from the Vivado post-implementation utilisation report for the placed-and-routed design, not from the HLS pre-synthesis estimate. The HLS estimate is only a prediction made before logic optimisation, placement and routing, and can deviate substantially from what the device actually consumes, so the post-implementation report is the only figure that reflects the real resource cost. All bitstreams reported in this chapter target a single PL clock: 200 MHz on the dense kernel and 150 MHz on the sparse kernel.

5.2 Dense Kernel: Latency, Resources, Energy and Scaling Analysis

This section compares the dense kernel's configurations against the CPU baseline on latency, energy, numerical accuracy and resource cost. The input matrix is always square ($n = m$), fully dense, and random.

5.2.1 Latency, Resources and Scaling

Five configurations are run on the same square test grid $n = m \in \{4, 8, 16, 32, 64, 128, 256\}$: software Gaussian elimination, software $LDL^\top$ Cholesky, hardware $LDL^\top$ Cholesky at $U = 8$, hardware Gaussian elimination at $U = 8$, and hardware $LDL^\top$ Cholesky at $U = 32$ (the largest unroll synthesised). All HW bitstreams use the Q31.40 fixed-point format.

Table 5.1: Dense solve wall-clock comparison. Times in ms. The last two columns are the speedup ratio of the SW baseline over HW Chol U32 (larger is better; values above 1 mean HW is faster).

$n = m$	SW Gauss	SW Chol	HW Chol U8	HW Gauss U8	HW Chol U32	$\frac{\text{SW Gauss}}{\text{HW Chol U32}}$	$\frac{\text{SW Chol}}{\text{HW Chol U32}}$
4	0.0014	0.0010	0.0138	0.183	0.0137	$0.11\times$	$0.08\times$
8	0.0084	0.0049	0.0228	0.361	0.0224	$0.38\times$	$0.22\times$
16	0.0588	0.0288	0.0474	0.726	0.0454	$1.30\times$	$0.63\times$
32	0.431	0.192	0.128	1.503	0.120	$3.6\times$	$1.6\times$
64	3.34	1.43	0.438	3.29	0.375	$8.9\times$	$3.8\times$
128	26.2	11.2	1.87	8.25	1.38	$19.0\times$	$8.1\times$
256	208	87.4	10.1	26.8	6.12	$34.0\times$	$14.3\times$

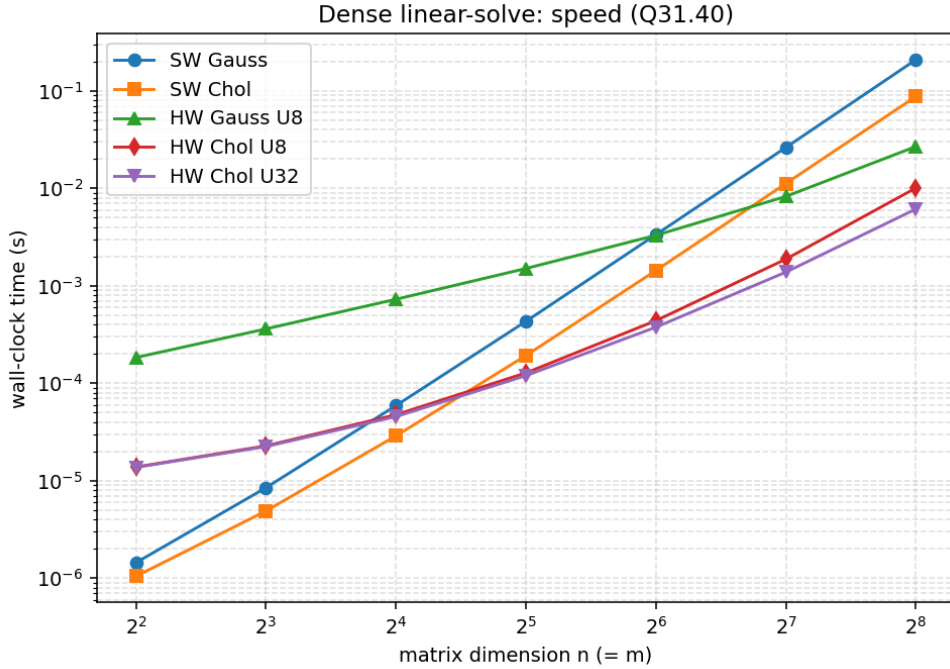


Figure 5.1: Per-solve wall-clock for the five dense configurations across the square sweep $n = m$, ZCU104 ARM-versus-FPGA (both axes are log-scaled).

HW Cholesky beats SW Gaussian past $n \approx 16$ and SW Cholesky past $n \approx 32$. At $n = 256$ the HW LDL^T Cholesky kernel at $U = 32$ finishes in 6.12 ms, i.e. $34.0\times$ faster than SW Gaussian and $14.3\times$ faster than the fair SW LDL^T baseline.

HW Cholesky also beats HW Gaussian at every size, by $2\text{--}3\times$ at $n = 256$. On symmetric-positive-definite matrices the Gaussian path carries partial pivoting and about double the FLOPs of LDL^T , and both costs turn into longer pipeline schedules in hardware. LDL^T Cholesky therefore dominates in both software and hardware, and is the algorithm of record for the rest of the chapter. One caveat: the HW Gaussian trace has a shallower slope than HW Cholesky on the

log-log plot, so the gap might narrow beyond $n = m = 256$ — but the ZCU104 resource budget did not allow testing that.

Table 5.2: FPGA resource cost of the three hardware configurations from Table 5.1. Counts from the Vivado post-place-and-route utilisation report; percentages relative to the XCZU7EV device totals (Table 2.2). The Resources column is the arithmetic mean of the five per-resource percentages. HW latency at $n = m = 256$.

Config	LUT	FF	BRAM	URAM	DSP	Resources	HW (ms)
HW Chol U8	40 934 (18 %)	57 197 (12 %)	20 (6 %)	32 (33 %)	274 (16 %)	17 %	10.1
HW Gauss U8	13 536 (6 %)	12 783 (3 %)	166 (53 %)	16 (17 %)	146 (8 %)	17 %	26.8
HW Chol U32	71 935 (31 %)	89 632 (19 %)	20 (6 %)	64 (67 %)	1 042 (60 %)	37 %	6.1

At equal unrolling factor, HW Cholesky is $2.6\times$ faster than HW Gaussian at $n = m = 256$ (10.1 ms vs 26.8 ms) for essentially the same device fraction (17 % mean for both). So the LDL^T advantage holds at equal cost, on energy as well as latency (Section 5.2.2). Raising the unroll factor to $U = 32$ buys a further $1.6\times$ over Chol U8 (6.1 ms) but more than doubles the footprint, to 37 %, driven by the DSP count growing to 1 042 (60 % of the device) and the URAM doubling to 64 tiles; the full resource-versus-unroll trade-off is in Section 5.2.4.

Kernel-internal phase split via ILA capture

At $n = m = 4$, the Cholesky solver at $U = 32$ still takes $13.7\ \mu\text{s}$, far more than software Cholesky ($1.0\ \mu\text{s}$). To see where that time goes, one `system_ila` (integrated logic analyzer, ILA) core was added in the Vivado block design on the kernel’s `master1` data bundle. It captures the `ARVALID` / `ARREADY` / `RVALID` / `RLAST` and write-channel handshakes at 16 384-sample depth — an $81.92\ \mu\text{s}$ window at the 200 MHz kernel clock, enough to hold the whole run at $n = m = 4$ and to capture the read or the write phase on its own at $n = m = 32$. The Vivado Hardware Manager was driven over JTAG, and the testbench used a single matrix per run, so each capture is exactly one kernel invocation.

Figures 5.2 to 5.4 show the three phases at $n = m = 4$, with all cycle counts measured from the trigger (rising `ARVALID`) at 5 ns per cycle. The kernel first issues three AR/R bursts on `master1`, one each for A , D , and b ; the last `RLAST` falls at cycle 167 ($0.835\ \mu\text{s}$ of read). Then (Figure 5.3) `master1` sits idle for 1 532 cycles ($7.66\ \mu\text{s}$) while K-formation and Cholesky run — the pure compute window. Finally (Figure 5.4) the output write is a single 5-cycle AW + W burst at cycle 1 699 (only 4 elements), and `BVALID` closes the run at cycle 1 728. The total is 1 728 cycles ($8.640\ \mu\text{s}$), split 9.7 % read / 88.7 % compute / 1.7 % write. The missing $5.1\ \mu\text{s}$ up to the $13.7\ \mu\text{s}$ wall-clock is probably host setup.

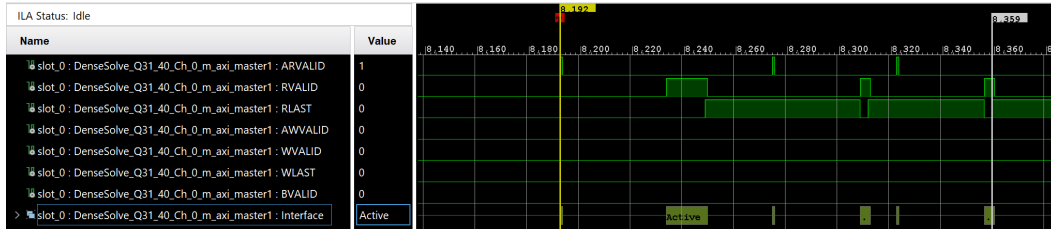


Figure 5.2: Read phase at $n = m = 4$. Three back-to-back AR/R bursts on master1 carry A , D , and b ; the last RLAST falls at cycle 167 (0.835 μ s).

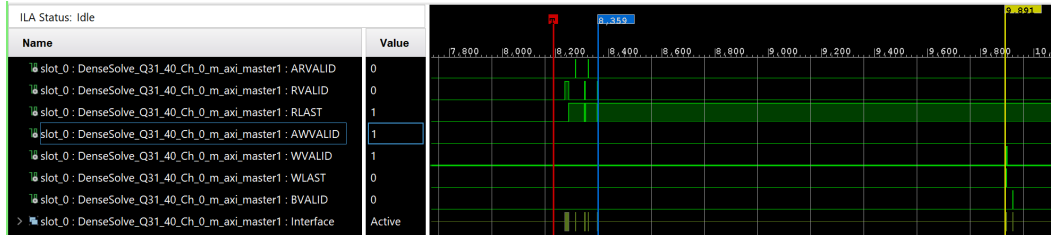


Figure 5.3: Compute window at $n = m = 4$: master1 is idle from cycle 167 to 1699, i.e. 1532 cycles or 7.66 μ s of pure compute between the last read response and the first write request.

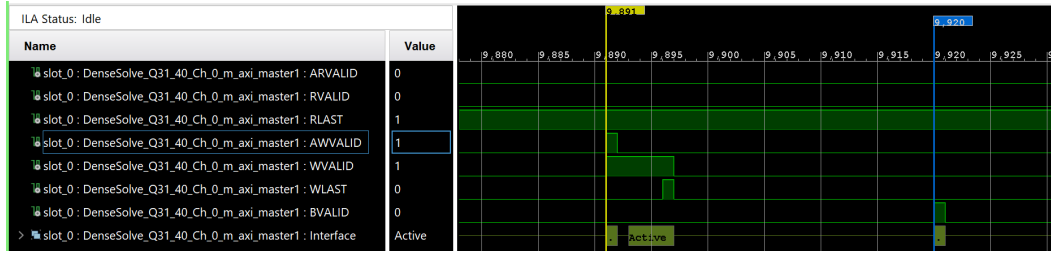


Figure 5.4: Write phase at $n = m = 4$: a 5-cycle AW + W burst writes the 4-element solution, and BVALID at cycle 1728 closes the run after 8.640 μ s total.

At $n = m = 32$ the run exceeds the 16384-sample (81.92 μ s) window, so it was captured in two separately-armed runs: one triggered on the first ARVALID (Figure 5.5), one re-armed to land on the write burst (Figure 5.6). The read phase alone now spans 2254 cycles (11.27 μ s), an order of magnitude longer than at $n = 4$ because the input grows as $m \cdot n + m + n$ words. The write phase stays a short single burst, since the output is still just m elements.

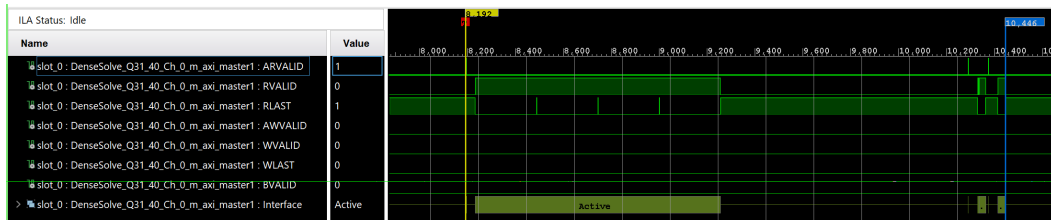


Figure 5.5: Read phase at $n = m = 32$ (separately-armed capture): the AR/R bursts now span 2254 cycles (11.27 μ s).

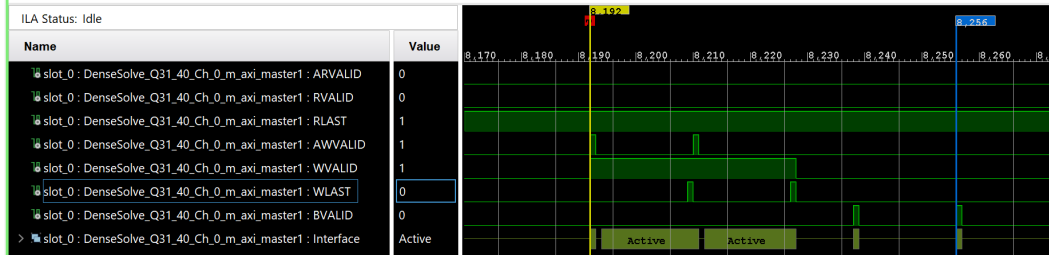


Figure 5.6: Write phase at $n = m = 32$, captured in a second armed window because the kernel duration exceeds the 16 384-sample depth of the data-side ILA.

These captures show the kernel is compute-bound, not transfer-bound: the read phase is only 9.7 % of the kernel time at $n = m = 4$, and stays a small fraction at $n = m = 32$ (the compute phase outruns the ILA window, so it has no exact figure, but the separate read and write captures make clear that compute dominates). The remaining gap to the full wall-clock latency is probably host setup and the `ap_start` / `ap_done` handshake.

5.2.2 Energy per Solve

The following table reports dynamic energy per solve, measured with libpmt [8] on the BOARD rail with idle subtraction per Equation 5.1. The same four configurations as the latency table are compared. Energies are in millijoules.

Table 5.3: Dense solve dynamic energy per call. Energies in millijoules. The last two columns are the energy-saving ratio of the SW baseline over HW Chol U32 (larger is better; values above 1 mean HW spends less energy).

$n = m$	SW Gauss	SW Chol	HW Chol U8	HW Gauss U8	HW Chol U32	$\frac{\text{SW Gauss}}{\text{HW Chol U32}}$	$\frac{\text{SW Chol}}{\text{HW Chol U32}}$
4	0.003	0.003	0.007	0.030	0.010	0.3×	0.3×
8	0.007	0.006	0.010	0.057	0.014	0.50×	0.43×
16	0.022	0.017	0.018	0.100	0.026	0.83×	0.65×
32	0.104	0.076	0.048	0.231	0.084	1.2×	0.91×
64	0.830	0.481	0.200	0.664	0.355	2.3×	1.4×
128	5.89	3.35	1.20	3.16	1.62	3.6×	2.1×
256	51.7	24.9	7.75	18.1	8.35	6.2×	3.0×

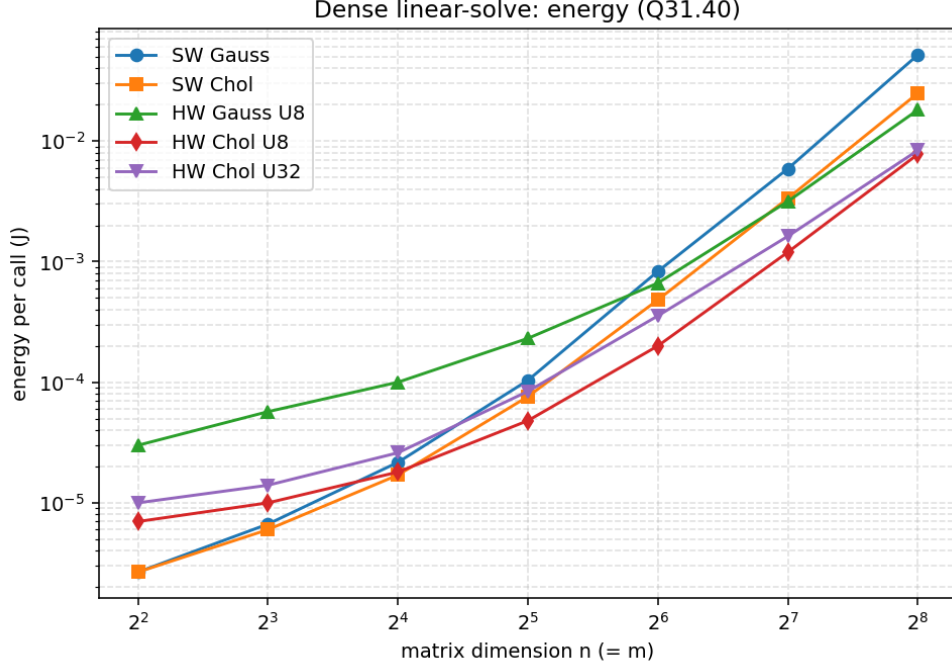


Figure 5.7: Dynamic energy per dense solve across the square sweep $n = m$, libpmt BOARD rail with idle subtraction (Equation 5.1).

At $n = m = 256$, HW Chol U32 uses $6.2\times$ less energy than SW Gaussian and $3.0\times$ less than SW Cholesky. Notably, HW Chol U8 draws slightly less energy than HW Chol U32, despite its higher latency. The energy savings are smaller than the matching speedups ($34.0\times / 14.3\times$ at U32, $20.7\times / 8.7\times$ at U8) because the FPGA fabric draws more power than the ARM cores when active: at $n = 256$, dynamic energy over elapsed time is ~ 1.4 W for HW Chol U32 (more parallel DSPs) and ~ 0.77 W for HW Chol U8, against ~ 0.25 W (SW Gaussian) and ~ 0.28 W (SW Cholesky). The energy gain therefore comes from the much shorter runtime, not from lower power. This also delays the crossover: HW wins on latency from $n \approx 32$ but on energy only from $n \approx 64$, because at small sizes the kernel runs too briefly for its higher active power to pay off.

5.2.3 Numerical Accuracy

The table below reports the signal-to-noise ratio (Equation 5.2) of the Q31.40 / $U = 8$ HW Cholesky kernel against the double-precision CPU reference. (The two references, Gaussian and Cholesky in doubles, agree to machine precision, so the choice does not affect the SNR.) The 31 integer and 40 fractional bits give enough headroom for the $K = ADA^\top$ accumulation at the tested sizes, with no overflow.

To check that the chosen algorithm does not lose accuracy, the same per-solve SNR is also measured for the HW Gaussian kernel at the identical Q31.40 / $U = 8$ point and the same four seeds (right-hand block of Table 5.4). Both use the same double-precision reference, so the

comparison isolates the per-solve fixed-point noise of the two algorithms.

Table 5.4: Numerical accuracy of HW Cholesky (Q31.40 fixed-point vs. double-precision CPU reference): four independent runs with different random matrix seeds (42, 41, 40, 39). The right-hand block repeats the same measurement for the HW Gaussian-elimination with the same seeds. “Avg” is the mean of the four runs at each size; the “Mean” row averages each column over all sizes.

$n = m$	HW Cholesky (dB)					HW Gaussian (dB)				
	Run 1	Run 2	Run 3	Run 4	Avg	Run 1	Run 2	Run 3	Run 4	Avg
4	186.6	187.6	211.4	223.6	202.3	186.2	176.2	216.1	224.0	200.6
8	197.6	133.7	138.3	172.3	160.5	206.5	127.6	138.9	172.3	161.3
16	150.9	167.3	174.1	183.0	168.8	149.9	171.9	173.2	181.6	169.2
32	175.9	180.4	181.7	145.0	170.8	176.1	179.5	186.8	162.7	176.3
64	158.6	172.8	173.9	117.0	155.6	168.7	173.6	159.2	120.5	155.5
128	122.2	138.8	108.7	134.2	126.0	137.2	148.9	112.5	140.7	134.8
256	124.4	123.1	125.7	136.8	127.5	130.1	119.5	131.2	143.1	131.0
Mean	159.5	157.7	159.1	158.8	158.8	165.0	156.7	159.7	163.6	161.2

Across the four runs, SNR stays above 100 dB at every tested size, so the accuracy is maintained.

Averaged over the four seeds, the two kernels track each other to within a few decibels at most sizes; the largest gap is about nine decibels in Gaussian’s favour at $n = m = 128$ (Cholesky 126.0 dB, Gaussian 134.8 dB). Neither holds a meaningful per-solve accuracy advantage at Q31.40: the overall means are 158.8 dB for Cholesky against 161.2 dB for Gaussian, a 2.4 dB gap. Choosing $LDL^\top$ Cholesky for its latency and energy advantage (Sections 5.2.1 and 5.2.2) therefore costs nothing in accuracy.

5.2.4 Unrolling-Factor Design Space

The dense unrolling factor (`K_BUILDING_UNROLLING_FACTOR` and `SOLVING_UNROLLING_FACTOR`, linked) is swept from 1 to 32 at the Q31.40 $LDL^\top$ Cholesky baseline. It controls array partitioning and loop unrolling in both the K -formation stage and the $LDL^\top$ factor and solve stage. The HW latency column reports the mean of $N = 20$ on-board calls at $n = m = 256$.

Table 5.5: Dense-kernel unroll sweep at Q31.40 Cholesky. Resources from the post-place-and-route Vivado utilization report, percentages are relative to the XCZU7EV device totals (Table 2.2). The Resources column is the arithmetic mean of the five per-resource percentages (aggregates the overall device occupancy). HW latency at $n = m = 256$.

U	LUT	FF	BRAM	URAM	DSP	Resources	HW (ms)
1	30 634 (13 %)	46 774 (10 %)	27 (9 %)	32 (33 %)	50 (3 %)	14 %	45.7
2	32 020 (14 %)	48 213 (10 %)	31 (10 %)	32 (33 %)	82 (5 %)	14 %	26.0
4	36 080 (16 %)	51 642 (11 %)	20 (6 %)	32 (33 %)	146 (8 %)	15 %	15.4
8	40 934 (18 %)	57 197 (12 %)	20 (6 %)	32 (33 %)	274 (16 %)	17 %	10.1
16	51 041 (22 %)	67 691 (15 %)	20 (6 %)	32 (33 %)	530 (31 %)	21 %	7.5
32	71 935 (31 %)	89 632 (19 %)	20 (6 %)	64 (67 %)	1 042 (60 %)	37 %	6.1

The two limiting resources are DSPs and URAMs. The DSP count grows linearly with U : the unrolling factor sets the number of parallel multipliers, and each maps one-to-one onto a DSP. The URAM count stays flat at 32 tiles up to $U = 16$, because until then the cyclic partition on the on-chip K matrix fits what a single URAM column can serve per cycle. At $U = 32$ the partition exceeds that limit, so a second URAM column is needed to feed the wider multiplier array and the count doubles to 64 tiles. This makes $U = 16$ a natural design point: it fully uses one URAM column without doubling it, delivers 7.5 ms at $n = m = 256$, and still leaves DSP headroom (530, 31 %) for other kernels on the same device.

The latency column shows that U has a large impact on latency, but the gain is not linear: some parts of the design are not unrolled and so do not benefit from the added parallelism.

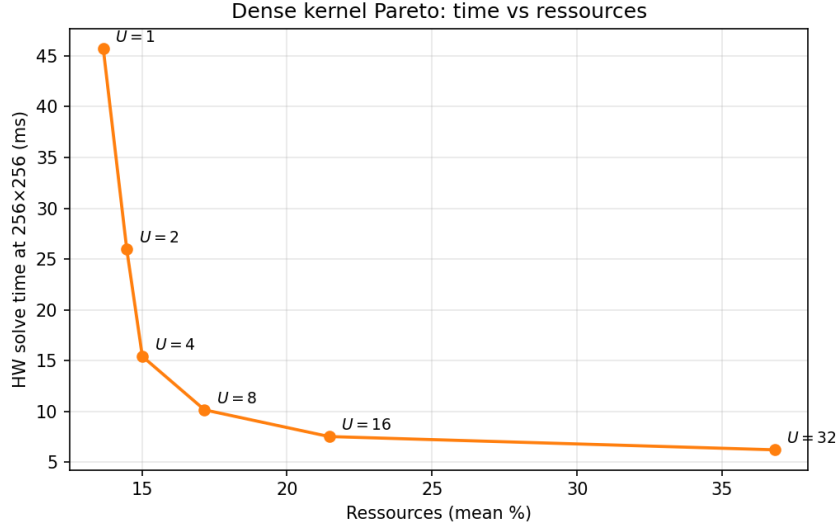


Figure 5.8: Dense-kernel Pareto: HW solve time at $n = m = 256$ versus resources (mean of LUT, FF, BRAM, URAM, DSP percentages), Q31.40 Cholesky baseline. Each point is a different unroll factor.

The plot above shows HW latency at $n = m = 256$ against the mean of the five resource percentages (LUT, FE, BRAM, URAM, DSP) over the unroll sweep. Every point lies on the Pareto front, since each unroll factor gives the best latency for its resource cost, but the curve flattens with diminishing returns as U grows. The $U = 8$ point is a good balance of latency and resource cost when other kernels must share the device.

Problem-Size Scaling at Fixed Unroll

$U = 16$ fills a single URAM column without doubling it (Section 5.2.4), which leaves the device's other URAM columns free. That raises a separate question: instead of spending more resources on parallelism, can the spare URAM hold a larger problem? At $U = 16$, raising the square size to $n = m = 384$ fills the URAM from 32 tiles (33 %) to all 96 (100 %), while the DSP count stays at 530 — the unroll factor, not the problem size, sets the multiplier count. The design still places, so the spare URAM buys a $2.25\times$ larger problem. The next power-of-two square, $n = m = 512$, does not fit: its two 512^2 on-chip arrays (A_d and K) would need about 128 URAM tiles against the 96 on the XCZU7EV, at any unroll factor. So $n = m = 384$ is the largest square this architecture admits, but only at $U = 16$ (it does not fit at $U = 32$). The price of the larger capacity is the lost $U = 32$ operating point, not the bigger array itself: at $n = m = 256$ this 384-capacity bitstream runs in 7.3 ms, matching the 7.5 ms of the native $U = 16$ build (Table 5.5), so the extra URAM costs essentially nothing at equal unroll. The $\sim 20\%$ gap is only against the native $U = 32$ build's 6.1 ms, the configuration the 384^2 arrays leave no room for.

Table 5.6: Square-size sweep at fixed $U = 16$ (Q31.40 Cholesky), all measured on a single bitstream built with `DENSE_MAX_M = DENSE_MAX_N = 384`.

$n = m$	Latency			Energy			SNR (dB)
	HW (ms)	SW (ms)	SW/HW	HW (mJ)	SW (mJ)	SW/HW	
256	7.27	87.4	$12.0\times$	8.50	24.9	$2.9\times$	124.4
384	19.64	293.2	$14.9\times$	25.49	121.4	$4.8\times$	125.5

Take-away: Dense kernel.

$LDL^\top$ Cholesky is the chosen dense algorithm: at equal unroll it beats Gaussian elimination on both latency and energy for the same device fraction, holds SNR above 100 dB at every size, and at $n = m = 256$ reaches a $34\times$ speedup and $6.2\times$ the energy efficiency of the ARM baseline. Three Pareto-optimal operating points emerge: $n = m = 256$ at $U = 32$ for the lowest latency (6.1 ms, at a steep 37 % mean occupancy); $n = m = 256$ at $U = 8$ for the best latency–resource balance (10.1 ms at only 17 %, leaving headroom for other kernels); and $n = m = 384$ at $U = 16$ for the largest square the XCZU7EV admits (full 96-tile URAM).

5.3 Sparse Kernel: Latency, Resources, Energy and Scaling Analysis

This section measures the SparseSolve kernel’s latency, energy, resources and scaling with problem size or density, against the equivalent CPU sparse LDL^T Cholesky solver. All numbers use the Q31.40 format and SPARSE_UNROLL=16.

5.3.1 Resource Footprint

Table 5.7: Post-place-and-route FPGA cost of the final sparse design (Q31.40, SPARSE_UNROLL = 16), next to the final dense bitstream (Chol $U = 32$, Table 5.2) for reference. Percentages relative to the XCZU7EV device totals (Table 2.2); the Resources column is the arithmetic mean of the five per-resource percentages.

Config	LUT	FF	BRAM	URAM	DSP	Resources
Dense Chol $U=32$	71 935 (31 %)	89 632 (19 %)	20 (6 %)	64 (67 %)	1 042 (60 %)	37 %
Sparse Q31.40 $U=16$	86 084 (37 %)	21 393 (5 %)	300 (96 %)	96 (100 %)	112 (6 %)	49 %

The two designs spend their silicon very differently. The sparse kernel is memory-bound: URAM is fully used (100 %) and BRAM sits at 96 %, while the DSPs are nearly idle at 6 %, against the dense kernel’s 60 %. Some parallelism opportunities are probably left in the sparse design; tuning it is left as future work (Section 6.4).

5.3.2 Latency, Energy and Scaling

The kernel is tested on the Unit-Commitment (UC) instances $1_T\{6, 12, 18, 24, 30, 36\}$, furnished as examples by the Research Center for Energy Networks [15]. Unit commitment is a standard power-systems problem: over a fixed horizon it decides which generators run and how much each produces, so that demand is met at every time step at minimum operating cost and other constraints are satisfied. The network is fixed throughout — 30 buses, 25 generators, 30 loads, 40 transmission lines — so only the horizon T (the number of hourly time steps) changes between instances. Both dimensions scale linearly with T : $m = 104 T$ constraints and $n = 200 T$ variables, with m from 624 to 3 744.

Table 5.8: Size and sparsity of the UC instances used in this section. m is the number of constraints, n the number of variables, A the $m \times n$ constraint matrix and $\rho_A = \text{nnz}(A)/(mn)$ its density.

Fixture	T	m	n	$\text{nnz}(A)$	ρ_A
1_T6	6	624	1 200	1 947	0.260 %
1_T12	12	1 248	2 400	3 969	0.133 %
1_T18	18	1 872	3 600	5 991	0.089 %
1_T24	24	2 496	4 800	8 013	0.067 %
1_T30	30	3 120	6 000	10 035	0.054 %
1_T36	36	3 744	7 200	12 057	0.045 %

Table 5.8 lists their size and sparsity. They are very sparse, and grow sparser as they get larger: the density ρ_A falls from 0.26 % at 1_T6 to 0.045 % at 1_T36. The reason is structural — m and n both grow with the horizon, while the non-zeros per row of A stay about constant.

The single-solve numbers come from one kernel solve, not a full IPM (Interior Point Method) run. One normal-equations system (a snapshot of (A, D, b) from an IPM iteration) is loaded once with `MODE_SETUP`, which caches the sparsity pattern on-chip and in DDR. The `MODE_SOLVE` call (kform, factorisation, triangular solve) is then repeated and averaged; that average is the reported per-iteration time and energy. `MODE_SETUP` is excluded from both, matching a real solve: the pattern is loaded once per LP, while `MODE_SOLVE` runs every iteration.

Per solve, the kernel narrows the gap with SW as the problem grows, but never closes it (Figure 5.9). HW is $1.25\times$ slower at 1_T6 (0.50 vs 0.40 ms), shrinking to $1.10\times$ at 1_T36 (3.10 vs 2.81 ms). On these very sparse UC systems the ARM sparse $LDL^\top$ stays ahead throughout: the FPGA never reaches per-iteration parity in latency or energy. Table 5.9 gives the per-fixture values.

Table 5.9: Per-iteration kernel solve time and energy on the UC fixtures, HW (SparseSolve, Q31.40) versus the ARM sparse $LDL^\top$ baseline. One `MODE_SOLVE` (kform + factorisation + triangular solve), with `MODE_SETUP` excluded.

Fixture	m	$\text{nnz}(L)/m$	time			energy		
			HW (ms)	SW (ms)	SW/HW	HW (mJ)	SW (mJ)	SW/HW
1_T6	624	3.30	0.50	0.40	$0.80\times$	5.81	4.61	$0.79\times$
1_T12	1 248	3.42	1.02	0.84	$0.82\times$	11.77	11.53	$0.98\times$
1_T18	1 872	3.46	1.54	1.28	$0.83\times$	17.73	17.41	$0.98\times$
1_T24	2 496	3.48	2.06	1.74	$0.85\times$	28.36	23.88	$0.84\times$
1_T30	3 120	3.49	2.58	2.26	$0.88\times$	35.28	30.69	$0.87\times$
1_T36	3 744	3.50	3.10	2.81	$0.91\times$	41.99	38.22	$0.91\times$

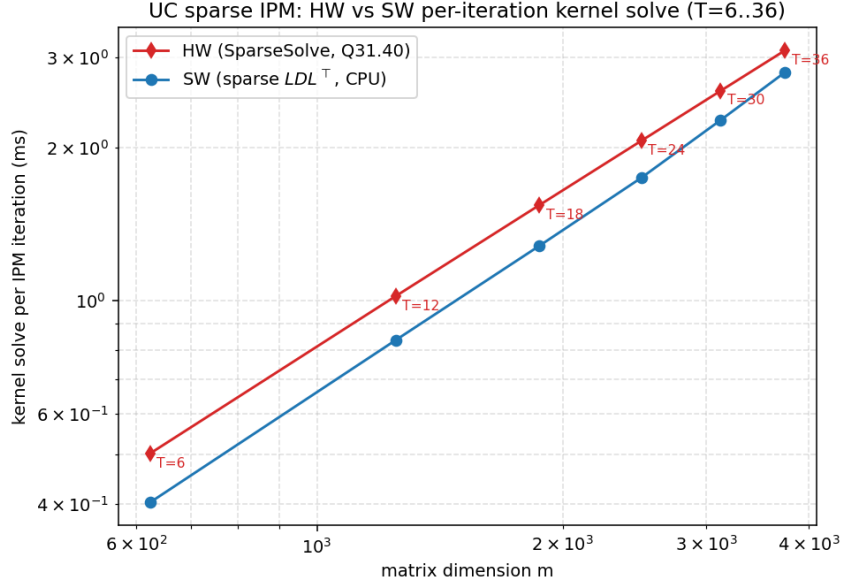


Figure 5.9: Per-iteration solve time of the SparseSolve kernel on UC fixtures $1_T\{6 \dots 36\}$, HW versus the ARM sparse LDL^T baseline, Q31.40. Measured as a single-solve microbenchmark (MODE_SOLVE only; the one-time MODE_SETUP is excluded). Both axes log-scaled.

Still, the kernel scales better than the CPU: the SW/HW time ratio improves from $0.80\times$ at 1_T6 to $0.91\times$ at 1_T36 , and the energy ratio from $0.79\times$ to $0.91\times$ over the same range. It does not catch the CPU, but it keeps up better as the problem grows.

Denser sparse structures: where the kernel wins

The kernel works mainly one column at a time (for example the COL_J loop in `hw/kernels/sparse_solve/src/sparse_factor.cpp`), with each iteration pipelined. More non-zeros per column let these column loops fill their pipeline better and hide the fixed overhead. The UC instances are very sparse, with only a few non-zeros per column in the factor L , so the kernel cannot fully exploit its pipelining on them.

To explore that, the kernel was run on extra fixtures: four synthetic banded systems with bandwidth 7 ($\text{nnz}(L)/m = 7$), four synthetic sparse systems with $\text{nnz}(L)/m$ from 15.9 to 97.8, and four synthetic dense (density and sparsity are complementary (density = $1 - \text{sparsity}$)) systems with $\text{nnz}(L)/m$ from 16.5 to 128.5. A banded matrix confines its non-zeros to a few diagonals on either side of the main diagonal. Cholesky factorisation adds no fill-in outside that band, so the factor L keeps the same bandwidth as the system matrix; its non-zeros per column stay capped by the bandwidth no matter how large m grows.

Table 5.10: Per-solve HW-vs-SW comparison on the synthetic square fixtures, Q31.40. SW is the CPU sparse LDL^T Cholesky.

Fixture	m	$\text{nnz}(L)/m$	HW (ms)	SW (ms)	Latency SW/HW	Energy SW/HW
banded_512	512	7.0	0.74	0.92	1.24	1.23
banded_1024	1 024	7.0	1.49	1.80	1.21	1.22
banded_1848	1 848	7.0	2.68	3.35	1.25	1.05
banded_4096	4 096	7.0	5.93	8.27	1.39	1.49
sparse_32	32	15.9	0.16	0.29	1.80	1.77
sparse_64	64	30.8	0.81	1.47	1.83	1.79
sparse_128	128	60.7	5.07	8.97	1.77	1.48
sparse_256	256	97.8	24.61	41.92	1.70	1.65
dense_32	32	16.5	0.20	0.41	2.04	1.98
dense_64	64	32.5	1.13	3.11	2.75	2.64
dense_128	128	64.5	7.42	24.53	3.30	3.05
dense_256	256	128.5	53.30	196.81	3.69	3.58

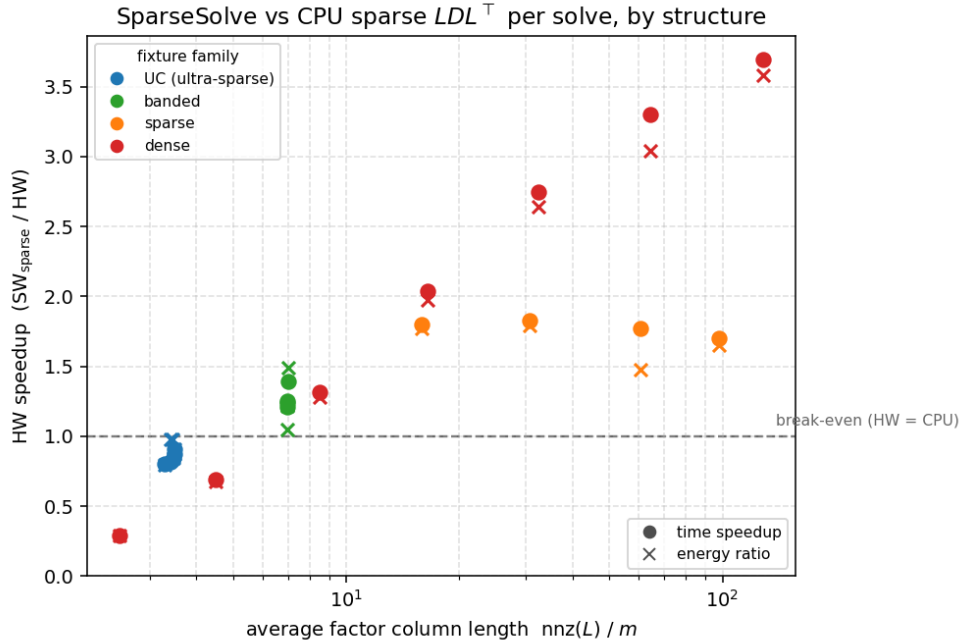


Figure 5.10: Per-solve HW-vs-SW ratio against average factor column length $\text{nnz}(L)/m$, across all fixture families (Q31.40, single `MODE_SOLVE`). Circles are the latency ratio, crosses the energy ratio; a point above the break-even line means HW wins. The horizontal axis is log-scaled; the vertical HW/SW ratio axis is linear.

The kernel beats the CPU on every one of these systems, by a margin that widens with column length. On the banded systems HW is already 1.21–1.39 $\times$ faster, the lead growing with size. Denser structure pushes it further ahead: the four sparse systems sit around 1.8 $\times$ (1.70–

$1.83\times$), and the dense systems climb from $2.0\times$ up to $3.69\times$ as their columns lengthen. The sparse systems stay flat rather than climb. Energy follows runtime: HW draws less on all of them, up to $3.6\times$ less on the dense systems.

5.3.3 Numerical Accuracy

The sparse kernel exposes the same per-solve SNR (Eq. 5.2) as the dense kernel. For each fixture the HW fixed-point solution $\hat{x}$ from a single `MODE_SOLVE` (kform, factorisation, triangular solve) is compared element-wise against a double-precision reference solution of the same normal-equations system. Table 5.11 reports the SNR at the Q31.40 baseline, across the UC fixtures and the synthetic banded, sparse, and dense systems of Table 5.10.

Table 5.11: Per-solve numerical accuracy of `SparseSolve` (Q31.40 fixed-point vs. a double-precision reference), one `MODE_SOLVE` per fixture.

Fixture	m	SNR (dB)
1_T6	624	187.6
1_T12	1 248	182.9
1_T18	1 872	188.3
1_T24	2 496	188.2
1_T30	3 120	185.2
1_T36	3 744	187.2
banded_512	512	209.8
banded_1024	1 024	210.3
banded_1848	1 848	210.4
banded_4096	4 096	210.6
sparse_32	32	185.8
sparse_64	64	180.6
sparse_128	128	155.6
sparse_256	256	143.5
dense_32	32	158.3
dense_64	64	130.5
dense_128	128	95.6
dense_256	256	114.8

On the real UC LP fixtures the kernel holds a machine-precision-class 183–188 dB independent of horizon length because the factor barely fills in ($\text{nnz}(L)/m \approx 3.3\text{--}3.5$, Table 5.10) and rounding does not accumulate as m grows. The synthetic families confirm the same law: the banded systems ($\text{nnz}(L)/m = 7$) stay ≥ 209 dB at every size, while the densifying sparse and dense families fall toward 144 dB and 96–115 dB as their columns lengthen.

Take-away: Sparse kernel on UC.

On the real UC LP fixtures the kernel never reaches per-iteration parity with the ARM sparse $LDL^\top$ baseline: its latency and energy are $1.10\text{--}1.25\times$ the CPU's across 1_T6–1_T36, though it scales better, narrowing the gap as the horizon grows without closing it. The cause is structural: UC factors carry only $\text{nnz}(L)/m \approx 3.3\text{--}3.5$ non-zeros per column, too few to fill the column-pipelined datapath. Where columns lengthen the kernel wins: $1.21\text{--}1.39\times$ on the banded systems, $\sim 1.8\times$ on the sparse family, and up to $3.69\times$ less latency and $3.6\times$ less energy on the dense family. Accuracy is never the limiting factor — the kernel holds a machine-precision-class 183–188 dB SNR on every UC fixture.

5.4 HW Kernel Selection: Dense-HW vs Sparse-HW Crossover

This section asks which HW kernel to use for a given problem. One kernel iteration solving is measured on the same square ($n = m$) random LP fed to both kernels. The dense kernel is the Q31.40 $LDL^\top$ Cholesky at $U = 32$ (max $m = n = 256$).

The sparse kernel runs on kform_paired fixtures over $m = n \in \{32, 64, 128, 256\}$ and $\rho \in \{0.05, 0.1, 0.2, 0.5, 1.0\}$, with the one-time MODE_SETUP excluded. The dense kernel forms a dense K regardless of the matrix, so its solve time depends only on m : it is reported flat across ρ .

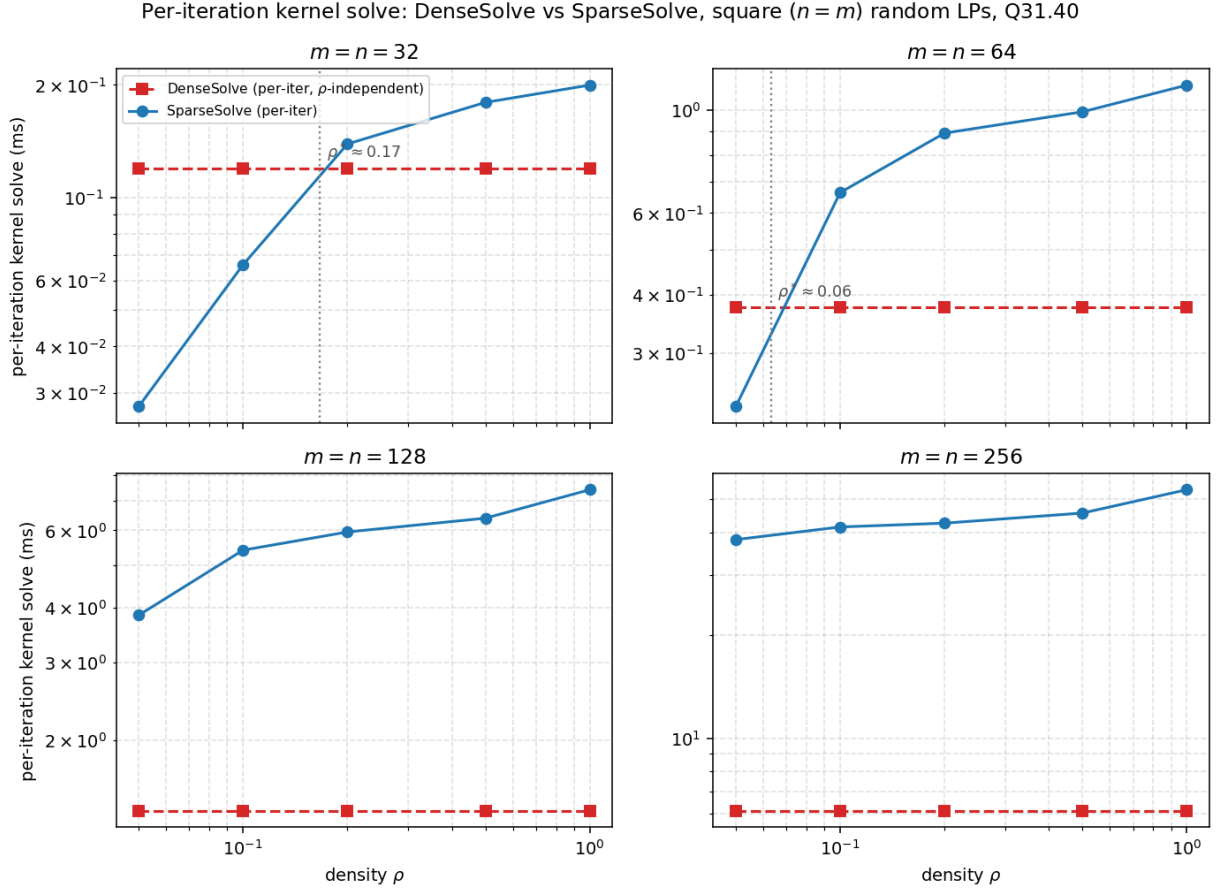


Figure 5.11: Per-iteration kernel solve time in function of density, HW DenseSolve versus HW SparseSolve, on shared square ($n = m$) random LPs, Q31.40. The dense trace is flat — the kernel densifies K regardless of ρ — while the sparse trace rises with ρ as K fills in.

Table 5.12: Per-iteration kernel solve time on the shared square ($n = m$) random-LP grid, mean over 6 seeds (ms). The dense kernel solve is ρ -independent (one value per m); the sparse kernel solve rises with ρ .

$m = n$	Dense (all ρ)	Sparse solve, by density ρ				
		0.05	0.1	0.2	0.5	1.0
32	0.120	0.028	0.066	0.139	0.180	0.200
64	0.375	0.230	0.664	0.891	0.990	1.131
128	1.38	3.85	5.41	5.95	6.40	7.42
256	6.12	38.1	41.5	42.5	45.5	53.3

The sparse solve rises with ρ as the K pattern and its Cholesky factor fill in: at $m = 256$ it climbs from 38.1 ms at $\rho = 0.05$ to 53.3 ms at $\rho = 1.0$. The density at which sparse stops being the faster kernel shrinks with size: at $m = 32$ the sparse solve is faster up to $\rho = 0.1$, at $m = 64$

only at $\rho = 0.05$, and at $m = 128$ and 256 the dense kernel is faster across the whole tested range ($\rho \geq 0.05$). The sparse kernel solves faster only when the problem is both small and very sparse; everywhere else the dense kernel wins.

The asymmetry has a structural cause, and Figure 5.11 shows its signature: the dense trace is flat while the sparse trace rises with ρ . The dense kernel’s schedule is fixed by the size m alone, every loop trip count is a compile-time constant and every access follows a regular stride, so it’s much easier to fully unroll, pipeline, and statically schedule a single datapath that is optimal for every input. The sparse kernel has no such guarantee: its loop trip counts (the non-zeros per column of K and of the factor L) and its indirect CSC/CSR gather accesses depend on the sparsity pattern of A , which is known only at runtime and changes from one problem to the next. HLS must therefore schedule conservatively for the worst case, and on short columns the pipeline drains faster than it fills.

Take-away: HW kernel selection.

On square random LPs the sparse kernel solves faster only at the smallest sizes and lowest densities — up to $\rho = 0.1$ at $m=32$, only at $\rho = 0.05$ at $m=64$. DenseSolve’s fixed $O(m^3/U)$ throughput wins everywhere else, by 6–9× per solve at $m = 256$. This is explained because the dense solver is well parallelised and pipelined, while the sparse solver was much harder to optimise.

5.5 End-to-End IPM

The previous sections measured single kernel solves in isolation. This section runs the *full* Interior-Point Method end to end with the FPGA factorisation and solving driving every iteration.

The experiment reuses the eighteen fixtures of Table 5.11: the real Unit-Commitment LPs and three synthetic families — banded, random-sparse, and random-dense — spanning $m = 32$ to 4096 . Two solvers run the *same* primal–dual IPM control loop on the ARM core (MAXITER = 50), differing only in how each iteration solves the normal equations $K = A \text{diag}(D) A^\top$: `sparse_hw` (the SparseSolve kernel at Q31.40 on the FPGA) and `sparse_sw` (the same sparse $LDL^\top$ in double precision on the ARM core). A run converges when the primal and dual infeasibilities and the duality measure $\mu = x^\top s/n$ all fall below `tol`.

Table 5.13: End-to-end IPM outcome across the eighteen fixtures of Table 5.11: iteration count (software vs. hardware SparseSolve, Q31.40), whether the hardware run converged ($\|r_p\|_\infty, \|r_d\|_\infty, \mu < \text{tol} = 10^{-4}$), and the end-to-end SNR of the final hardware solution against the software reference (contrast with the per-solve SNR of Table 5.11).

Fixture	m	SW it.	HW it.	HW conv.	SNR (dB)	SW (ms)	HW (ms)	Latency SW/HW
1_T6	624	27	50	no	6.1	20.7	—	—
1_T12	1 248	34	50	no	2.5	54.0	—	—
1_T18	1 872	37	50	no	2.0	92.3	—	—
1_T24	2 496	38	50	no	2.0	136.1	—	—
1_T30	3 120	38	50	no	1.3	181.2	—	—
1_T36	3 744	39	50	no	1.1	231.9	—	—
banded_512	512	6	6	yes	158.1	6.3	8.2	0.77
banded_1024	1 024	6	6	yes	158.1	12.8	16.2	0.79
banded_1848	1 848	6	6	yes	157.9	24.8	29.0	0.86
banded_4096	4 096	6	6	yes	157.9	62.7	66.1	0.95
sparse_32	32	11	11	yes	122.7	3.4	2.6	1.31
sparse_64	64	12	12	yes	127.5	17.5	10.8	1.62
sparse_128	128	13	13	yes	109.2	112.8	65.0	1.74
sparse_256	256	15	15	yes	95.8	592.2	353.2	1.68
dense_32	32	6	6	yes	144.6	2.3	1.9	1.21
dense_64	64	6	6	yes	120.3	16.5	8.1	2.04
dense_128	128	6	6	yes	78.1	126.0	45.0	2.80
dense_256	256	6	6	yes	71.2	998.0	300.0	3.33

The three synthetic families are well-conditioned and converge in the same iteration count as software. The real UC instances converge on none: the hardware reaches MAXITER on all six, while software needs 27–39 iterations. The failure is cumulative: each Newton step leaves an $O(10^{-6})$ fixed-point error until the accumulating error overwhelms the step.

Where it converges, the hardware is faster end to end for sparse and dense fixtures (but not the banded). Because both solvers take the same number of iterations on every converged fixture, this end-to-end SW/HW latency ratio follows the per-solve latency ratio of Table 5.10: the dense family reaches a $3.3\times$ end-to-end speedup, approaching the $3.69\times$ per-solve latency ratio as the solve grows to dominate the fixed host overhead.

Take-away: End-to-end IPM.

On the synthetic dense, sparse and banded fixtures the fixed-point IPM converges in the same iteration count as software and, where the solve is heavy, wins end to end (up to $3.3\times$); on the real UC instances the error compounds and it reaches MAXITER. Implementing a mixed-precision iterative refinement would be a next step to address this limitation (Section 6.4).

5.6 FEN Urban-Energy Case Study

In this section, a multi-energy investment-planning problem from the *FEN* (Research Center for Energy Networks, ETH Zürich) [15] is analyzed, of the kind produced by the renewable-energy-hub optimiser REHO [20]. All runs in this section execute on the host workstation (AMD Ryzen 7 8845HS, 16 threads).

The FEN model is a two-stage investment-planning LP, which is why the instance comes as two matrices rather than one. The first stage sizes the energy system once; the second operates it over the horizon. A ($m \times n_x$) holds the operational constraints and B ($m \times n_y$) the investment-coupling columns.

The horizon comes at six temporal aggregations: averaging weather years in groups of twelve leaves one representative year (period_12Year, the smallest model), while no aggregation keeps all twelve (period_1Year, the full-resolution model). The model grows with the number of representative years. Table 5.14 lists the resulting dimensions.

Table 5.14: FEN instance dimensions across the six temporal aggregations, ordered from smallest to largest.

Aggregation	Rep. years	m	n	ρ	nnz $[A \mid B]$ ($\times 10^6$)	72-bit store (MB)
period_12Year	1	3 635 244	1 734 591	1.2×10^{-6}	7.66	68.9
period_6Year	2	7 270 488	3 469 182	7.0×10^{-7}	17.66	158.9
period_4Year	3	10 905 732	5 203 773	5.3×10^{-7}	30.01	270.1
period_3Year	4	14 540 976	6 938 364	4.4×10^{-7}	44.71	402.4
period_2Year	6	21 811 464	10 407 546	3.6×10^{-7}	81.15	730.3
period_1Year	12	43 622 928	20 815 092	2.7×10^{-7}	246.81	2 221

The constraint matrix is extremely sparse: the density of $[A \mid B]$ stays between 10^{-6} and 10^{-7} . At the project’s 72-bit fixed-point width, the non-zeros alone cost tens to hundreds of megabytes; the full-resolution model would need over 2 GB just for the non-zero values (the index arrays add a further ≈ 12 bytes per nonzero).

Gurobi [17] solves the smallest instance in under a minute. Presolve is decisive: it removes 92.9 % of the rows, collapsing the 7.27 M-row model (the 3.64 M two-sided constraints split into separate rows) to 517 846 rows and 594 904 columns before a single barrier step. The barrier is then the critical path: it converges in 59 iterations but takes 24 s, 59% of the total 41 s runtime.

Table 5.15: Gurobi 13.0.1 barrier solve of `period_12Year` on the host CPU (16 threads, `Method=2`). Times are the solver-internal phase durations; host model assembly is excluded. The optimal objective is 9.0756×10^{12} .

Phase	Time (s)	Detail
Presolve	5.5	7.27 M $\rightarrow$ 517 846 rows, 1.48 M nnz
Ordering	0.4	AMD fill-reducing
Barrier (59 iter)	24.0	
Crossover	10.9	132 195 simplex iterations
Total (Runtime)	40.8	

How much of this is the interior-point algorithm, and how much is Gurobi’s engineering? To separate the two, the same instance was solved with the open-source HiGHS IPM solver [18], both on the full model and on Gurobi’s presolved system. HiGHS reaches the same optimum but is far slower: 1 806 s on the full model, and still 879 s on Gurobi’s 517 846-row presolved system — a $44\times$ and $21\times$ gap against Gurobi’s ≈ 41 s (Table 5.16). Both run the same interior-point method, so the gap is engineering, not algorithm: Gurobi is a remarkably optimised solver, hard to beat.

Table 5.16: Solver comparison on `period_12Year`, host CPU, single run. All three reach the same optimum 9.0756×10^{12} .

Solver	Configuration	Solve (s)	IPM iter
Gurobi 13.0.1	barrier, full model	41	59
HiGHS 1.13	IPM, full model	1 806	70
HiGHS 1.13	IPM, on Gurobi-presolved (518 k)	879	64

Take-away: FEN urban-energy instance.

Result: Gurobi presolve cuts the smallest FEN LP from 3.64 M constraints to 517 846 rows; the barrier solves it in 59 iterations / ≈ 41 s on the host CPU. The presolved system is still $63\times$ the `SparseSolve` build ceiling and $>10^3\times$ the dense kernel’s $n=256$ build ceiling.

Lesson: The FPGA kernels are a backend for the on-chip barrier inner solve of small, dense LPs. Large-scale energy planning is out of reach for the on-chip kernel as it stands, but cutting the FEN down with presolve or decomposition could expose subproblems the FPGA can accelerate.

Chapter 6

Conclusion

6.1 Summary of Contributions

This project set out to locate the computational bottleneck of an MILP solver, accelerate at least two of its critical kernels on an FPGA through High-Level Synthesis, explore the design space of each for Pareto-optimal operating points, and evaluate the result against a CPU baseline on latency, energy, and accuracy. Profiling placed the bottleneck in the inner LP solve: the matrix factorization and solving inside each Interior-Point iteration dominates the runtime (Chapter 2). Two HLS kernels address it. The `DenseSolve` kernel builds the matrix, factorizes it in place with an LDL^T Cholesky, and back-substitutes, all in on-chip URAM under a DATAFLOW pipeline; at $n = m = 256$ it solves $34\times$ faster than the ARM Gaussian baseline ($14.3\times$ against the same-algorithm reference) for $6.2\times$ less energy, and holds its signal-to-noise ratio above 100 dB across the size sweep. The `SparseSolve` kernel runs a symbolic-then-numerical sparse Cholesky and stores the factor L in CSR/CSC format; it beats the CPU on banded and moderately dense systems (up to $3.7\times$ faster, $3.6\times$ leaner) but stays behind on the very sparse Unit-Commitment instances, where the short factor columns starve its pipeline.

Two further contributions complete the picture. The design space of each kernel was swept over unrolling factor, memory placement, and fixed-point Q-format: three Pareto-optimal operating points emerge for the dense kernel (fastest, best latency–resource balance, and largest square that fits the XCZU7EV). The kernels were then integrated into the full IPM and run end to end. On well-conditioned problems the IPM converges in the same iteration count as software, and where the inner solve is heavy the FPGA wins end to end (up to $3.3\times$); on ill-conditioned instances, such as the real UC LPs, the per-iteration error compounds and the solver stalls. Two urban-energy case studies bound the reach: the UC fixtures, which the kernel factorizes faithfully but whose IPM does not yet converge at fixed point, and the FEN instance, which was profiled but whose presolved half-million-row barrier system sits $63\times$ above the sparse kernel’s on-chip ceiling.

6.2 Dissemination

An intermediate version of this work was presented on a poster at the *Sustainability Conference for Responsible Research Computing* (SC4RC 2026), held at CERN in May 2026. The part of the poster dedicated to this project covered the dense Cholesky `DenseSolve` kernel and its design-space exploration on the ZCU104, including the cost–latency Pareto front and the resulting $\approx 34\times$ speedup over the board’s CPU. Presenting it was a great opportunity to better understand the whole REHO project and its context, and to share my work with the REHO team and other

researchers, while discovering a lot of other interesting projects.

6.3 Findings and Lessons Learned, Self-Assessment

The FPGA acceleration paid off where the problem stayed small and dense. The on-chip LDL^T kernel solved the 256×256 normal-equations system $14.3\times$ faster than the same algorithm in software and $34\times$ faster than the naive Gaussian baseline (Section 5.2.1). That speedup is real, but the dense kernel is not the kernel that matters most at scale for this application: energy planning problems (like the FEN case study) are large and extremely sparse, far beyond the $m = n = 256$ that fits on-chip. The sparse kernel is the one that matters at scale: it beat the CPU on banded and moderately dense systems (up to $3.7\times$ faster, $3.6\times$ leaner) but stayed behind on the very sparse unit-commitment instances — their columns are so sparse that the pipeline starves. The deeper reason it was harder to optimize at all is structural (Section 5.4): the dense kernel’s cost is fixed by the problem size, whereas the sparse kernel’s cost depends on the matrix it is given, so no single static schedule can be optimal. For the dense solver the bottleneck could be read directly from the Vitis HLS report; for the sparse solver it could not, because many loops depend on the data.

Two practical things made progress slow. The iteration loop was long: every change had to be synthesized remotely and pushed to the board before it could be measured, so testing on the board rather than in simulation alone often cost hours per experiment. Finding the true bottleneck was also harder than expected: Vitis HLS reports worst-case iteration counts, but those do not show where the pipeline actually stalls on real data, so the latency had to be reasoned out indirectly.

Looking back, the algorithmic and mathematical work was the hardest part of the project for me. I should perhaps have tried to understand the problem better before jumping straight into the implementation, and I should have asked my supervisors for help more often. I also focused too much on the dense kernel when that time might have gone to the sparse one, the one the application actually needs. It is a little frustrating not to have had the time to push the sparse kernel further, but I am very happy with the dense-kernel results, and the analysis did give a better understanding of the problem structure and the design trade-offs, hopefully enough to guide future work on the sparse kernel. Overall, I am satisfied with the project and the results, and I learned a lot about FPGA design and optimization along the way.

6.4 Future Work

Several directions remain:

- **Solve several Branch-and-Bound nodes at once.** Rather than speed up a single LP solve (Section 2.1), solve several at once: the depth-first search always holds several independent

pending nodes, and three copies of the DenseSolve kernel fit on the XCZU7EV (at $U = 8$ it uses only 33 % of the URAM). However, the gain would stay below $3\times$, as on multi-core CPUs (Figure 2.2): evaluating nodes ahead of the search solves some LPs that a sequential depth-first traversal would have pruned, so part of that work is wasted.

- **Fit larger problems.** The factor L sits entirely on-chip, capping the problem size well below FEN scale (Section 3.4.3). A stronger fill-reducing ordering (AMD, approximate minimum degree [6] for example), streaming values from DDR, or a larger Zynq UltraScale+/Versal board would each raise the ceiling.
- **Decompose the problem into smaller ones.** The FEN model (profiled in Section 5.6) is hourly (8760 time steps per year, about 400 operational constraints per step), so cutting the time-coupling with budgeting variables, in the manner of Jacobson et al. [21] (Section 2.7.2), would reduce each subproblem, perhaps enough to fit in kernel range. Time did not permit implementing this, but it would be a route to planning scale.
- **Make the fixed-point IPM converge.** The accelerator’s binding limitation is convergence (Section 5.5): the small fixed-point error in each solve accumulates over the IPM iterations until the solver diverges. The most direct fix is *mixed-precision iterative refinement* [11]: keep the factorization in fixed point on the FPGA, but correct each step in double precision on the host with matrix regularization or a finer inner-solve format as lighter alternatives.
- **Rethink the linear algebra.** A matrix-free iterative solver (conjugate gradients or PDHG, primal–dual hybrid gradient) would avoid forming and factorizing $K = A \text{diag}(D) A^\top$ altogether, sidestepping fill-in, but at the cost of more iterations, and therefore probably a finer fixed-point precision to converge.
- **Remove polling.** Replace the driver’s busy-wait polling with an interrupt, so the ARM core can sleep while the kernel runs, and save energy.

Bibliography

- [1] Tobias Achterberg. “Constraint Integer Programming”. Open access: <https://depositonce.tu-berlin.de/handle/11303/1931>. PhD thesis. Technische Universität Berlin, 2007. DOI: 10.14279/depositonce-1634.
- [2] AMD/Xilinx. *UltraScale Architecture DSP Slice User Guide (UG579)*. <https://docs.amd.com/v/u/en-US/ug579-ultrascale-dsp>. 2024.
- [3] AMD/Xilinx. *Vitis High-Level Synthesis User Guide (UG1399)*. <https://docs.amd.com/r/en-US/ug1399-vitis-hls>. 2024.
- [4] AMD/Xilinx. *ZCU104 Evaluation Board User Guide (UG1267)*. <https://docs.amd.com/v/u/en-US/ug1267-zcu104-eval-bd>. 2024.
- [5] AMD/Xilinx. *Zynq UltraScale+ Device Technical Reference Manual (UG1085)*. <https://docs.amd.com/v/u/en-US/ug1085-zynq-ultrascale-trm>. 2024.
- [6] Patrick R. Amestoy, Timothy A. Davis, and Iain S. Duff. “An approximate minimum degree ordering algorithm”. In: *SIAM Journal on Matrix Analysis and Applications* 17.4 (1996), pp. 886–905. DOI: 10.1137/S0895479894278952.
- [7] Arm Limited. *Arm Cortex-A53 MPCore Processor Technical Reference Manual (DDI 0500)*. <https://developer.arm.com/documentation/ddi0500/>. 2016.
- [8] ASTRON. *PMT: Power Measurement Toolkit*. <https://git.astron.nl/RD/pmt>. 2024.
- [9] Samuel Bayliss, Christos-Savvas Bouganis, George A. Constantinides, and Wayne Luk. “An FPGA Implementation of the Simplex Algorithm”. In: *International Conference on Field-Programmable Technology (FPT)*. 2006, pp. 49–56. DOI: 10.1109/FPT.2006.270294.
- [10] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Open access: <https://web.stanford.edu/~boyd/cvxbook/>. Cambridge University Press, 2004.
- [11] Erin Carson and Nicholas J. Higham. “Accelerating the Solution of Linear Systems by Iterative Refinement in Three Precisions”. In: *SIAM Journal on Scientific Computing* 40.2 (2018), A817–A847. DOI: 10.1137/17M1140819.
- [12] Yanqing Chen, Timothy A. Davis, William W. Hager, and Sivasankaran Rajamanickam. “Algorithm 887: CHOLMOD, Supernodal Sparse Cholesky Factorization and Update/Downdate”. In: *ACM Transactions on Mathematical Software* 35.3 (2008), 22:1–22:14. DOI: 10.1145/1391989.1391995.
- [13] E. Cuthill and J. McKee. “Reducing the bandwidth of sparse symmetric matrices”. In: *Proceedings of the 24th National Conference of the ACM*. 1969, pp. 157–172. DOI: 10.1145/800195.805928.
- [14] Timothy A. Davis. *Direct Methods for Sparse Linear Systems*. Fundamentals of Algorithms. SIAM, 2006.
- [15] FEN, ETH Zürich. *FEN: Research Center for Energy Networks*. <https://www.fen.ethz.ch/>.

- [16] Gurobi Optimization, LLC. *Advanced Algorithms in Mathematical Optimization (Paris, October 2022)*. Presentation slides; parallel MIP benchmark reproduced therein from Achterberg and Wunderling, “Mixed Integer Programming: Analyzing 12 Years of Progress” (2013). Available at <https://fr.scribd.com/document/921176421/2022-10-Paris-Advanced-Algorithms>. 2022.
- [17] Gurobi Optimization, LLC. *Gurobi Optimizer Reference Manual*. <https://www.gurobi.com/documentation/>. 2024.
- [18] HiGHS contributors. *HiGHS – High Performance Software for Linear Optimization*. <https://highs.dev/>. Accessed: 2026-06-05. 2025.
- [19] Carl Ingemarsson and Oscar Gustafsson. “Hardware Architecture for Positive Definite Matrix Inversion Based on LDL Decomposition and Back-Substitution”. In: *Asilomar Conference on Signals, Systems, and Computers*. 2016, pp. 859–863. DOI: 10.1109/ACSSC.2016.7869171.
- [20] IPESE Laboratory, EPFL. *REHO: Renewable Energy Hub Optimizer*. <https://reho.readthedocs.io/en/main/>.
- [21] Anna Jacobson, Filippo Pecci, Nestor Sepulveda, Qingyu Xu, and Jesse Jenkins. *A Computationally Efficient Benders Decomposition for Energy Systems Planning Problems with Detailed Operations and Time-Coupling Constraints*. Preprint; published in *INFORMS Journal on Optimization* 6(1):32–45, 2024. <https://arxiv.org/abs/2302.10037>. 2023. DOI: 10.1287/ijoo.2023.0005. arXiv: 2302.10037 [math.OC].
- [22] A. Rodriguez-Garcia, L. Pizano-Escalante, R. Parra-Michel, O. Longoria-Gandara, and J. Cortez. “Fast fixed-point divider based on Newton-Raphson method and piecewise polynomial approximation”. In: *International Conference on Reconfigurable Computing and FPGAs (ReConFig)*. 2013, pp. 1–6. DOI: 10.1109/ReConFig.2013.6732291.
- [23] Yusuke Shimai, Junichi Tani, Hiroki Noguchi, Hiroshi Kawaguchi, and Masahiko Yoshimoto. “FPGA Implementation of Mixed Integer Quadratic Programming Solver for Mobile Robot Control”. In: *International Conference on Field-Programmable Technology (FPT)*. 2009, pp. 447–450. DOI: 10.1109/FPT.2009.5377635.
- [24] Samuel Williams, Andrew Waterman, and David Patterson. “Roofline: An Insightful Visual Performance Model for Multicore Architectures”. In: *Communications of the ACM* 52.4 (2009), pp. 65–76. DOI: 10.1145/1498765.1498785.

Appendix A

Declaration on the use of generative AI

Some generative-AI tools were used during this project, including Claude Code and Gemini. These tools were used for:

- LaTeX writing assistance: example of prompt used for this purpose: "Rephrase the following paragraph to correct grammar/orthography and improve clarity, while preserving the technical content".
- Helping to understand and implement the mathematical formulations and algorithms.
- Helping to write and refactor redundant code, such as testbench codes, deployment scripts, plotting scripts...
- Debugging: AI tools were used to help identify and fix bugs in the code, by providing suggestions based on error messages and code context.

All AI-generated content was critically evaluated by the author, and any suggestions were manually reviewed and edited as needed to ensure accuracy and relevance to this project.

Appendix B

Project Repositories

B.1 GitHub

The codebase of this project is available on GitHub at <https://github.com/esl-epfl/milp-fpga>. The README file provides an overview of the repository, and gives instructions for usage. The repository is private, please contact the owner Denisa-Andreea Constantinescu for access.

B.2 Google Drive

A shared Google Drive folder was used during the project to store documents: <https://drive.google.com/drive/folders/1iQu8STWx4kEN4aeTy04MruPhv1wRHmYl>. The folder is shared privately, please contact Denisa-Andreea Constantinescu for access. It contains:

- `Weekly_meetings.pptx` – slide deck used during weekly supervisor meetings, tracking progress and discussion points.
- `Project_description.pdf` and `Project_description.docx` – the official project description (Appendix C).
- `Planning.xlsx` and `Planning_v2.xlsx` – initial and revised project planning.
- `Doc/` – documents related to the project.
- `Diagrams/` – diagrams.

Appendix C

Project Description

FPGA Acceleration for Urban Digital Twin Models: Optimizing Mixed-Integer Linear Programming (MILP) for Energy Efficiency

Project Objectives

The motivation of this project is to design and develop novel energy-efficient hardware accelerators for solving MILP problems on FPGA platforms. For fast prototyping, we will adopt a hardware-software co-design approach to explore novel domain-specific accelerators using High Level Synthesis. By leveraging the parallelism and reconfigurability of FPGAs, the goal is to achieve significant energy savings while maintaining or improving the computational performance of MILP solvers.

Obligatory Objectives

1. **Understand and Describe MILP Kernels:**
 - Conduct an in-depth analysis of key MILP algorithms, focusing on the most computationally intensive kernels.
 - Identify the parts of the MILP solving process that can benefit most from hardware acceleration.
2. **Implement and Optimize MILP Kernels on FPGA Using High-Level Synthesis (HLS):**
 - Design and implement at least two critical kernels of the MILP solving process using HLS tools.
 - Focus on optimizing these kernels for energy efficiency, resource usage, and latency.
3. **Evaluate and Explore Design Trade-offs:**
 - Conduct a comprehensive design space exploration to evaluate the trade-offs between energy consumption, computational latency, resource usage, throughput and hyperparameters of the solver.
 - Present at least three Pareto optimal solutions for each implemented kernel, demonstrating the balance between these metrics.
4. **Experimental Evaluation on FPGA:**
 - Test and validate the designed accelerators on a selected FPGA platform.
 - Measure and compare energy consumption, execution time, and solution accuracy against traditional CPU-based implementations.
 - [Optional] Experimental validation of the proposed designs on UrbanTwin MILP problems, showcasing the advantages of FPGA-based acceleration.

Additional Objectives

1. **Integration with Full MILP Solver Pipeline:**
 - Integrate the optimized MILP kernels into a complete MILP solver pipeline on FPGA.
 - Evaluate the performance of the integrated system in terms of energy efficiency and computational throughput.
2. **Qualitative Analysis of Solution Accuracy:**
 - Assess the accuracy and quality of the solutions provided by the FPGA-accelerated MILP solver.
 - Compare the results with those obtained from traditional software-based solvers, focusing on metrics such as solution optimality and robustness.

Required Knowledge and Skills

- Proven experience in hardware description languages such as VHDL or Verilog.
- Experience with Xilinx FPGAs or Intel FPGAs and associated development tools.
- Strong programming skills in Python, C/C++, and familiarity with High-Level Synthesis (HLS) tools.
- Understanding of optimization techniques, particularly MILP.

Type of Work

- **Theoretical Analysis (30%):** Involves understanding MILP algorithms, energy-efficient design principles, and FPGA architecture.
- **Design and Experimentation (70%):** Focuses on the implementation, optimization, and evaluation of the MILP kernels on FPGA.

This project will be conducted under the supervision of experts in hardware design and optimization from the Embedded Systems Laboratory (ESL) and Industrial Process and Energy Systems Engineering (IPESE), providing the student with an opportunity to contribute to cutting-edge research in energy-efficient computing.

References

- Renewable Energy Hub Optimizer (REHO): <https://reho.readthedocs.io/en/main/>
- MILP with Gurobi (commercial): <https://www.gurobi.com/resources/chapter-1-why-mixed-integer-programming-mip/>
- HiGHS (open source): <https://highs.dev/>
- Vitis HLS User Guide: <https://docs.amd.com/r/en-US/uq1399-vitis-hls>

Lab: ESL

Sections: SEL

Supervisor: Denisa-Andreea Constantinescu, Rubén Rodríguez Álvarez, Ana Catarina Gouveia Braz, Cédric Terrier, David Atienza Alonso

Contact email: denisa.constantinescu@epfl.ch; ruben.rodriquezalvarez@epfl.ch; catarina.braz@epfl.ch; cedric.terrier@epfl.ch; david.atienza@epfl.ch